

AWS MESSAGING & SECRETS MANAGEMENT

Complete Hands-On Implementation Guide

Amazon SQS • Amazon MQ • AWS Secrets Manager

Document Type	Hands-On Lab + Technical Reference + Implementation Documentation
AWS Region	us-east-1 (use the actual Region used during the lab if different)
Audience	DevOps / Cloud / AWS learners and implementation teams
Primary Goal	Understand, configure, test, secure, monitor and document SQS, Amazon MQ and Secrets Manager
Document Date	22 September 2026
Status	Implementation-ready training document

Document safety: Never paste real passwords, access keys, private keys, database credentials or secret values into screenshots, source code, Git repositories or this document. Use lab-only credentials and redact sensitive values.

Table of Contents

1. Executive Summary
2. Objectives
3. Scope and Lab Standards
4. AWS Environment and Prerequisites
5. Service Selection at a Glance
6. Amazon SQS — Concepts, Architecture and Limits
7. SQS Hands-On Lab — Standard Queue
8. SQS Hands-On Lab — FIFO Queue
9. SQS Hands-On Lab — Dead-Letter Queue
10. SQS Security, IAM, Encryption and Monitoring
11. Amazon MQ — Concepts, Architecture and Limits
12. Amazon MQ Hands-On Lab — RabbitMQ Broker
13. Amazon MQ Security, Networking and Troubleshooting
14. AWS Secrets Manager — Concepts, Architecture and Limits
15. Secrets Manager Hands-On Lab
16. Secrets Manager IAM, KMS, Rotation and Monitoring
17. Integrated Reference Architecture
18. Security and Production Best Practices
19. Troubleshooting Runbook
20. Validation and Evidence Checklist
21. Cost and Cleanup Checklist
22. Interview / Viva Questions and Answers

- 23. 23. Final Observations
- 24. 24. Conclusion
- 25. 25. Official AWS References

1. Executive Summary

This document provides a complete, structured implementation guide for three AWS services used in modern distributed applications: Amazon Simple Queue Service (Amazon SQS), Amazon MQ, and AWS Secrets Manager. It combines service fundamentals with console-based hands-on implementation, security, networking, encryption, monitoring, quotas, troubleshooting, evidence collection and cleanup.

The services solve different problems. Amazon SQS provides AWS-native message queuing and application decoupling. Amazon MQ provides managed broker technologies such as ActiveMQ and RabbitMQ for workloads that depend on traditional messaging protocols and broker semantics. AWS Secrets Manager securely stores, retrieves and manages sensitive application information such as database credentials and API keys.

Core distinction: SQS = managed AWS-native queue; Amazon MQ = managed traditional message broker; Secrets Manager = managed secret lifecycle and secure retrieval service.

2. Objectives

- Understand the purpose, architecture, use cases and core terminology of Amazon SQS, Amazon MQ and AWS Secrets Manager.
- Perform hands-on implementation using the AWS Management Console and verify the configuration.
- Understand authentication, authorization, IAM least privilege, encryption, KMS, networking and monitoring.
- Understand important service limits, quotas, retention periods, payload sizes, throughput considerations and regional behavior.
- Test normal and failure scenarios such as SQS visibility timeout and dead-letter processing.
- Capture screenshots and observations suitable for an implementation submission.
- Document production-oriented best practices, common mistakes, troubleshooting steps and cost controls.

3. Scope and Lab Standards

3.1 Scope

- Amazon SQS Standard queue.
- Amazon SQS FIFO queue, ordering, message groups and deduplication.
- Amazon SQS visibility timeout, long polling, retention and dead-letter queue.
- SQS IAM permissions, encryption and CloudWatch monitoring.
- Amazon MQ for RabbitMQ broker, VPC/subnet/security-group concepts and broker connectivity.
- Amazon MQ security, protocols, monitoring, quotas and cost considerations.
- AWS Secrets Manager secret creation, retrieval, IAM, KMS, versions, rotation concepts and CloudTrail auditing.
- Combined reference architecture for an application using queues, a broker and centralized secrets.

3.2 Lab Standards

- Use a dedicated lab naming convention such as devops-demo-*
- Record the AWS Region and exact service configuration actually used.
- Use IAM roles instead of long-lived access keys for EC2/Lambda/ECS workloads whenever possible.
- Restrict security-group rules to required source security groups/CIDRs and required ports.
- Do not expose Amazon MQ management or application ports to 0.0.0.0/0 unless there is a documented, justified requirement.
- Do not store real production credentials in this lab.
- Delete chargeable resources after the lab is complete.

4. AWS Environment and Prerequisites

Item	Recommended / Example	Record Actual Value
AWS Region	us-east-1	_____
AWS Account	Training / lab account	_____
Operating system	Windows + Git Bash / Linux	_____
AWS CLI	Current supported version	_____
VPC	Existing lab VPC or dedicated lab VPC	_____
EC2 client	Optional for Amazon MQ / IAM testing	_____
KMS	AWS managed key or customer-managed key as required	_____

4.1 Before You Start

26. Sign in to the AWS Management Console.
27. Confirm the Region shown in the console matches the Region intended for the lab.
28. Confirm the AWS account and billing environment are correct.
29. Confirm you have permission to create SQS queues, Amazon MQ brokers, Secrets Manager secrets, IAM policies/roles and related monitoring resources.
30. If using Amazon MQ, identify a VPC, subnet(s) and security group suitable for the broker.
31. Create a screenshot folder with subfolders: SQS, Amazon-MQ and Secrets-Manager.
32. Keep a lab log containing timestamp, configuration, test result and screenshot filename.

5. Service Selection at a Glance

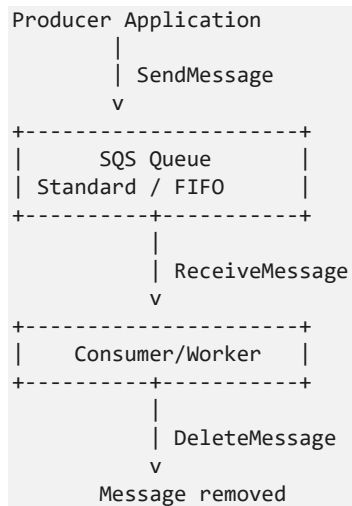
Requirement	SQS	Amazon MQ	Secrets Manager
Primary purpose	Asynchronous message queuing	Managed traditional message broker	Secure secret storage and lifecycle
Typical application	AWS-native distributed workloads	Existing broker/protocol-dependent workloads	Applications needing credentials/API keys
Main technologies	AWS SQS APIs	ActiveMQ / RabbitMQ and supported protocols	Secrets Manager APIs / SDKs
Ordering	FIFO available	Depends on broker/implementation	Not applicable
Dead-lettering	Native DLQ support	Broker-specific	Not applicable
Rotation	Not applicable	Broker-specific credentials	Supported secret rotation
VPC focus	AWS service endpoints / optional VPC endpoints	Strong VPC/subnet/security-group focus	AWS service; VPC endpoint can be used for private connectivity
Primary security model	IAM + queue policy + encryption	IAM for AWS APIs + broker authentication + network controls + TLS	IAM + resource policy + KMS

6. Amazon SQS — Concepts, Architecture and Limits

6.1 What is Amazon SQS?

Amazon SQS is a fully managed message queuing service used to decouple application components. A producer sends a message to a queue; a consumer receives and processes it and normally deletes it after successful processing.

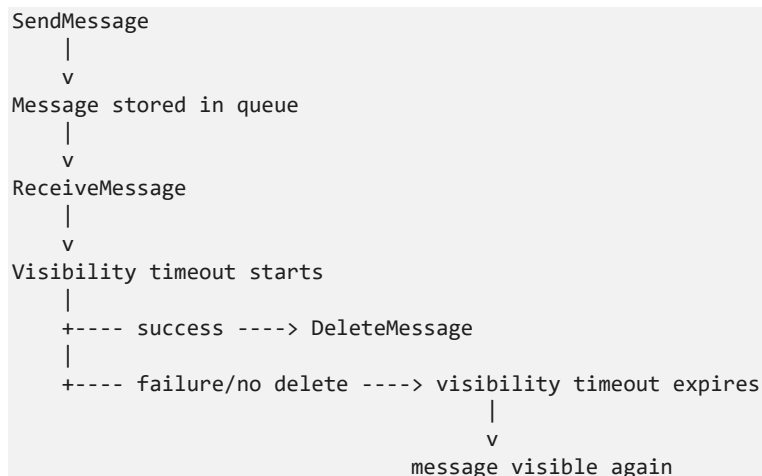
6.2 Basic Architecture



6.3 Standard vs FIFO

Characteristic	Standard	FIFO
Ordering	Best-effort; no ordering guarantee	Ordered processing within a message group
Delivery model	At-least-once delivery	Designed for ordered, deduplicated processing
Name	Normal queue name	Queue name must end in .fifo
Use cases	High-scale asynchronous workloads where ordering is not required	Transactions/workflows where ordering and deduplication matter
Parallelism	Highly scalable	Use multiple message groups for parallelism

6.4 Message Lifecycle



6.5 Important SQS Concepts

- Visibility timeout: the period during which a received message is hidden from other consumers. Current documented range: 0 seconds to 12 hours; default is 30 seconds.
- Message retention: configurable from 1 minute through 14 days; default is 4 days.
- Message size: maximum 1 MiB.
- Long polling: ReceiveMessage can wait up to 20 seconds for messages, reducing empty responses.

- Delay: a message can be delayed for up to 15 minutes.
- Batch APIs: SQS supports batches of up to 10 messages for supported send/receive/delete operations.
- Dead-letter queue: failed messages can be moved after a configured maximum receive count.
- Encryption: SQS supports server-side encryption; AWS-managed and KMS-backed options are available depending on the configuration.
- IAM: use least-privilege actions such as SendMessage for producers and ReceiveMessage/DeleteMessage for consumers.

6.6 SQS Important Limits / Quotas

Limit / Feature	Current documented value / concept	Operational note
Message size	1 MiB	Use S3/object storage plus a pointer for larger payloads.
Retention	1 minute to 14 days; default 4 days	SQS is not a permanent database.
Visibility timeout	0 seconds to 12 hours; default 30 seconds	Set longer than normal processing time, with margin.
Long polling	Up to 20 seconds	Useful for reducing empty receives.
Delay	Up to 15 minutes	Useful for short deferred processing.
Message attributes	Up to 10	Validate application design if many attributes are needed.
Batch size	Up to 10 messages	Batching can improve efficiency.
Standard in-flight	Approximately 120,000	Quota/behavior can vary; monitor backlog and in-flight messages.
FIFO throughput	Depends on Region, batching and high-throughput FIFO configuration	Check current regional quota before production design.

Quota rule: AWS service quotas can change. For production architecture, verify the current quota page for the exact Region and queue type instead of copying an old tutorial value.

7. SQS Hands-On Lab — Standard Queue

7.1 Objective

Create a Standard SQS queue, send and receive a message, demonstrate visibility timeout behavior, delete the message, and record evidence.

7.2 Console Navigation

33. Open the AWS Management Console.
34. Search for Amazon SQS and open the service.
35. Confirm the intended AWS Region.
36. Select Queues and choose Create queue.
37. Select Standard.
38. Enter the queue name: devops-demo-queue.
39. For the first lab, leave retention at the default 4 days unless your training requirement specifies another value.
40. Set visibility timeout to 30 seconds for the initial test. You can later change it to 60 seconds for a clearer demonstration.
41. Set delivery delay to 0 seconds.
42. Enable long polling and use 20 seconds for the lab if the console exposes the queue-level receive wait-time setting.
43. Keep server-side encryption enabled.
44. Review and choose Create queue.

7.3 Send a Test Message

45. Open devops-demo-queue.
46. Choose Send and receive messages.
47. Enter the following test payload.
48. Choose Send message.

```
{
  "orderId": "ORD1001",
  "customer": "TrainingUser",
  "amount": 2500,
  "status": "NEW"
}
```

7.4 Receive and Delete

49. Choose Poll for messages.
50. Open the received message and verify the body and message metadata.
51. Do not delete the message immediately if you are demonstrating visibility timeout.
52. After the demonstration, receive the message again if required and choose Delete.
53. Poll again and verify the message is no longer available.

7.5 Visibility Timeout Demonstration

54. Send one test message.
55. Receive it but do not delete it.
56. Immediately poll again. The message may not appear because it is temporarily invisible.
57. Wait until the visibility timeout expires.
58. Poll again. The message should become available again.
59. This demonstrates why consumers must delete successfully processed messages and why visibility timeout must be aligned with processing time.

7.6 Evidence

Screenshot ID	Capture
SQS-01	SQS console and Region
SQS-02	Standard queue created
SQS-03	Queue configuration
SQS-04	Message sent
SQS-05	Message received
SQS-06	Visibility timeout behavior
SQS-07	Message deleted / queue after deletion

8. SQS Hands-On Lab — FIFO Queue

8.1 Objective

Create a FIFO queue and demonstrate queue naming, message-group ordering and deduplication behavior.

8.2 Configuration

60. Choose Create queue.
61. Select FIFO.
62. Use a name ending in .fifo, for example devops-demo-orders.fifo.
63. Enable content-based deduplication for the lab if appropriate.
64. Keep encryption enabled.

65. Create the queue.

8.3 Ordering Test

66. Send message Order 1 with MessageGroupId customer-1.
67. Send message Order 2 with MessageGroupId customer-1.
68. Send message Order 3 with MessageGroupId customer-1.
69. Receive the messages and record the order returned.
70. Repeat with a second MessageGroupId to understand that message groups are the unit used for ordered sequencing and can be used to increase parallelism.

FIFO design rule: Do not assume that simply choosing FIFO makes the whole distributed application globally sequential. Ordering is enforced within a message group. Choose MessageGroupId according to the business entity whose order must be preserved.

9. SQS Hands-On Lab — Dead-Letter Queue

9.1 Objective

Create a DLQ and configure a source queue so repeatedly failing messages can be isolated for investigation.

71. Create a Standard queue named devops-demo-dlq.
72. Open devops-demo-queue and locate its dead-letter queue / redrive configuration.
73. Select devops-demo-dlq as the destination.
74. Set Maximum receives to 3 for a lab demonstration.
75. Send a test message.
76. Receive the message repeatedly without deleting it, allowing the visibility timeout to expire between attempts.
77. After the configured receive threshold is reached, verify that the message is moved to the DLQ according to the configured redrive policy.
78. Record the message and approximate receive behavior.

FIFO compatibility: A FIFO source queue should use a FIFO dead-letter queue; a Standard source queue should use a Standard dead-letter queue.

10. SQS Security, IAM, Encryption and Monitoring

10.1 Least-Privilege IAM

Role	Typical minimum actions
Producer	sqs:SendMessage; add only required queue-read/attribute actions if the application needs them
Consumer	sqs:ReceiveMessage, sqs:DeleteMessage; add GetQueueAttributes when required
Administrator / lab operator	Queue management actions as required; avoid using administrator permissions for application runtime

10.2 Encryption

Use server-side encryption. For more control over key policies, auditing and key lifecycle, a customer-managed KMS key can be used where appropriate. When SSE-KMS is used, producers/consumers and integrated services may require appropriate KMS permissions in addition to SQS permissions.

10.3 CloudWatch Monitoring

- `ApproximateNumberOfMessagesVisible` — backlog waiting to be processed.
- `ApproximateNumberOfMessagesNotVisible` — messages currently in-flight/under visibility timeout.
- `ApproximateAgeOfOldestMessage` — useful indicator of processing delay.
- `NumberOfMessagesSent / Received / Deleted` — useful for traffic and processing behavior.
- Create alarms around backlog and oldest-message age according to application SLOs.

11. Amazon MQ — Concepts, Architecture and Limits

11.1 What is Amazon MQ?

Amazon MQ is a managed message broker service supporting Apache ActiveMQ and RabbitMQ. It is intended for applications that use broker technologies and supported messaging protocols, particularly where migrating to an AWS-native queue would require substantial application changes.

11.2 When Amazon MQ Fits

- Existing ActiveMQ or RabbitMQ applications need a managed AWS deployment.
- Applications depend on established broker protocols or client libraries.
- A migration strategy benefits from keeping broker semantics while moving infrastructure to AWS.

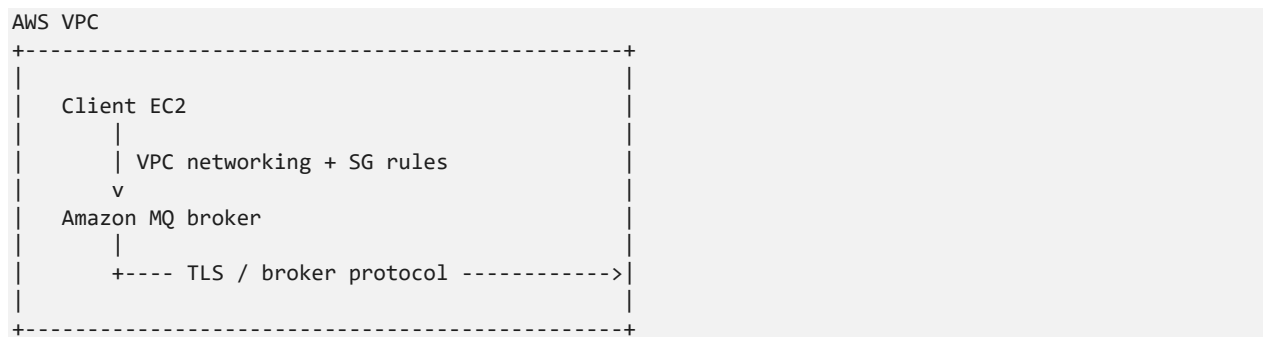
11.3 When SQS May Be the Architectural Choice Instead

- A new AWS-native application needs straightforward asynchronous decoupling.
- The application does not require traditional broker protocols or broker-specific features.
- The team wants an AWS-native queue without managing broker instances and broker networking.

11.4 Amazon MQ Engines

Engine	Key concept
ActiveMQ	Traditional enterprise broker; commonly used with Java/JMS and supported messaging protocols.
RabbitMQ	Widely used open-source broker with AMQP and a broad client ecosystem.

11.5 Networking Model



11.6 Deployment and Availability

Amazon MQ deployment choices depend on engine and Region. Active/standby ActiveMQ deployments place broker instances in different Availability Zones for higher availability. Always verify the current console options for the selected engine and Region.

11.7 Amazon MQ Quotas

Area	Important point
Broker count	AWS documents a regional broker quota; verify the current quota for your Region/account.
Connections	Limits vary by engine and broker instance size.
Storage	Depends on broker configuration/engine and selected storage options.
Throughput	Depends on broker instance type, protocol, workload and topology.
Engine versions	Supported versions change; select from versions currently offered in the console.

Version rule: Do not copy an engine version from an old lab manual. Record the version actually offered and selected on the day of implementation.

12. Amazon MQ Hands-On Lab — RabbitMQ Broker

12.1 Objective

Create a small RabbitMQ broker, configure VPC networking and authentication, obtain its endpoint, and prepare it for a controlled connectivity test.

12.2 Create the Broker

79. Open AWS Management Console and search for Amazon MQ.
80. Choose Create broker.
81. Select RabbitMQ.
82. Choose a small lab-appropriate broker instance type that is currently offered in your Region.
83. Choose a deployment mode supported by the selected engine/instance. Use single-instance for a basic lab if available; use a multi-AZ/HA option for an availability exercise if required and budgeted.
84. Enter a broker name such as devops-rabbitmq-lab.
85. Create a strong lab username and password. Do not use a real production credential.
86. Select the VPC and subnet configuration appropriate for the lab.
87. Select or create a dedicated security group.
88. Enable appropriate logging if available.
89. Review all settings and choose Create broker.
90. Wait until the broker status is RUNNING.

12.3 Security Group Design

- Permit only the broker protocol/TLS ports required by the chosen engine and client.
- Prefer source security-group references when the client is an EC2 instance in the same VPC.
- Avoid 0.0.0.0/0 for broker application/management access in a training architecture unless explicitly required for a controlled test.
- Document each inbound rule: source, protocol, port and purpose.

12.4 Connectivity Test Plan

91. Launch or identify an EC2 client in a network path that can reach the broker.
92. Verify the broker status is RUNNING.
93. Resolve the broker endpoint from the client if DNS testing is appropriate.
94. Verify the security group permits the required connection from the client.
95. Use a RabbitMQ-compatible client/application with TLS and the correct credentials.

96. Publish a test message and consume it using the client/application.
97. Record the successful endpoint, protocol, test message and result without recording the password.

12.5 Evidence

Screenshot ID	Capture
MQ-01	Create broker / engine selection
MQ-02	Broker configuration and deployment mode
MQ-03	VPC/subnet/security group
MQ-04	Broker RUNNING
MQ-05	Endpoint information with sensitive details redacted where necessary
MQ-06	CloudWatch/monitoring evidence
MQ-07	Client connectivity/message test

13. Amazon MQ Security, Networking and Troubleshooting

13.1 Security Layers

- IAM controls AWS API/console management actions.
- Broker-level authentication controls access to messaging endpoints.
- TLS protects supported client-to-broker communications.
- Security groups control network reachability.
- Private subnets can reduce internet exposure.
- CloudTrail can provide auditing of AWS API activity; CloudWatch provides operational monitoring.

13.2 Troubleshooting Order

98. Confirm the broker is RUNNING.
99. Confirm the client is in the expected Region/VPC/network path.
100. Confirm the endpoint and protocol match the broker engine.
101. Confirm DNS resolution from the client.
102. Check security-group inbound/outbound rules.
103. Check network ACLs and route tables if applicable.
104. Verify username/password.
105. Verify TLS configuration and certificate trust.
106. Check application/client logs for authentication, TLS or protocol errors.
107. Check CloudWatch metrics/logs for broker-side symptoms.

14. AWS Secrets Manager — Concepts, Architecture and Limits

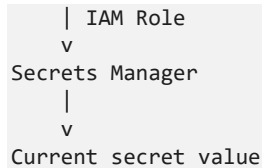
14.1 What is Secrets Manager?

AWS Secrets Manager is a managed service for storing, retrieving, controlling and rotating sensitive information such as database credentials, API keys, tokens and application secrets.

14.2 Bad vs Good Pattern

```
BAD:
application.properties
db.password=RealPassword
```

```
GOOD:
Application
|
```



14.3 Secret Structure Example

```
{
  "username": "appuser",
  "password": "<REDACTED>",
  "host": "database.example.com",
  "port": "3306",
  "database": "companydb"
}
```

14.4 Encryption

Secrets Manager encrypts secret values at rest using AWS Key Management Service. The AWS managed key for Secrets Manager is suitable for many normal scenarios. A customer-managed KMS key can provide additional control over key policy, permissions and lifecycle when required.

14.5 Versions and Rotation

Secrets Manager maintains secret versions. Common staging labels include `AWSCURRENT` and `AWSPREVIOUS`. Rotation changes credentials according to the configured rotation mechanism and helps reduce the lifetime of static credentials.

14.6 Regional Behavior

Secrets are regional resources. Secrets can be replicated to other AWS Regions when supported and appropriate, which can help multi-Region applications and disaster-recovery designs.

14.7 Important Limits

Feature	Current documented value / concept	Operational note
Secret value size	Up to 65,536 bytes	Do not use Secrets Manager as general-purpose object storage.
Secret name	1–512 characters; supported characters include letters/numbers and selected punctuation	Use clear hierarchical names.
API usage	Service quotas apply to API request rates	Cache secrets in applications where appropriate.
Version creation	Excessive PutSecretValue/UpdateSecret calls can create many versions	Avoid repeatedly creating versions more frequently than needed.

Sensitive-data rule: Never place secret values in tags, source repositories, screenshots, CloudWatch logs, shell history or application logs.

15. Secrets Manager Hands-On Lab

15.1 Objective

Create a lab secret, retrieve it securely, inspect its metadata/version information and prepare a least-privilege IAM role for application retrieval.

15.2 Create Secret

108. Open AWS Console and search for Secrets Manager.
109. Choose Store a new secret.
110. Select Other type of secret for a generic lab secret.
111. Create key/value pairs such as username=appuser, password=<LAB-ONLY-PASSWORD>, environment=lab.
112. Choose the AWS managed Secrets Manager KMS key for the basic lab, or a customer-managed key for the advanced KMS exercise.
113. Use a name such as devops/lab/database.
114. Add non-sensitive tags such as Environment=Lab and Project=DevOps.
115. Do not put passwords in tags.
116. For the first lab, leave automatic rotation disabled.
117. Choose Store.

15.3 Retrieve Secret in Console

118. Open the secret.
119. Choose Retrieve secret value.
120. Verify the key/value pairs.
121. Capture a screenshot only after masking/redacting the password and any other sensitive value.

15.4 CLI Retrieval

```
aws secretsmanager get-secret-value \  
  --secret-id devops/lab/database \  
  --region us-east-1
```

CLI security: Do not put real secret values directly into command-line arguments. Command history, process listings or logs can expose command-line content. Prefer IAM roles and secure application retrieval.

15.5 Evidence

Screenshot ID	Capture
SM-01	Secrets Manager console
SM-02	Secret creation configuration
SM-03	Secret stored / metadata
SM-04	Secret value screen with password fully redacted
SM-05	IAM policy/role allowing only required secret access
SM-06	KMS configuration if customer-managed key is used
SM-07	Version information
SM-08	Rotation configuration if implemented
SM-09	CloudTrail evidence of API activity

16. Secrets Manager IAM, KMS, Rotation and Monitoring

16.1 Least-Privilege IAM Policy Pattern

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Effect": "Allow",  
      "Action": [  
        "secretsmanager:GetSecretValue"
```

```

    ],
    "Resource": "<SECRET-ARN>"
  }
]
}

```

Use the exact secret ARN whenever practical rather than granting access to every secret in the account.

16.2 EC2 Retrieval Architecture



16.3 Rotation

122. Select a supported secret type and rotation method.
123. Configure the required rotation schedule and dependencies.
124. For supported AWS-managed database scenarios, use the managed rotation capability where appropriate.
125. For custom rotation workflows, use the documented Lambda-based rotation pattern where supported.
126. Test rotation in a lab before production use.
127. Confirm the application can retrieve the new current version and that the old credential is no longer unnecessarily usable.

16.4 Caching

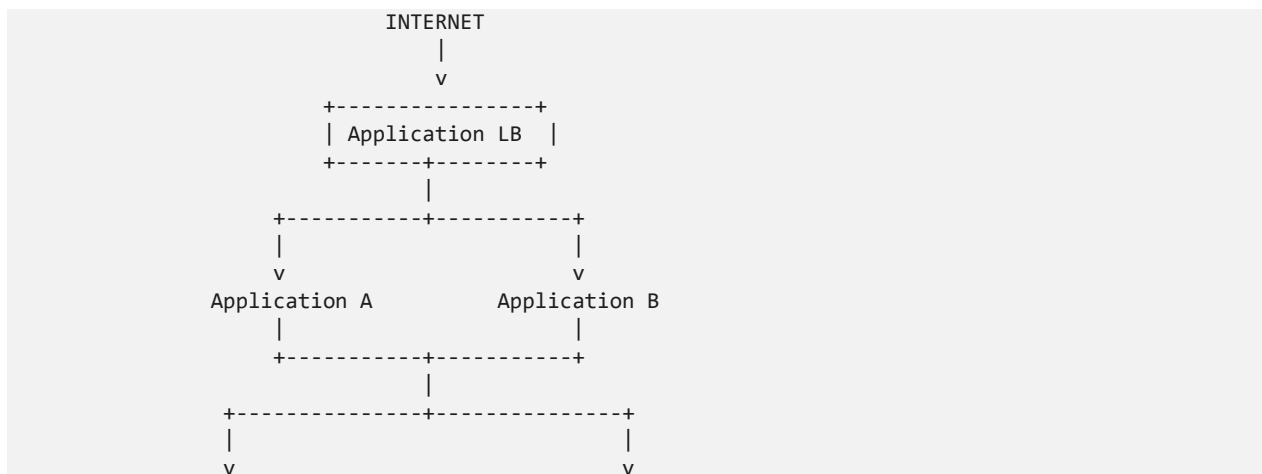
Applications should avoid calling GetSecretValue for every request at high volume. Use an appropriate client-side cache with a refresh policy that balances availability, rotation responsiveness and API cost.

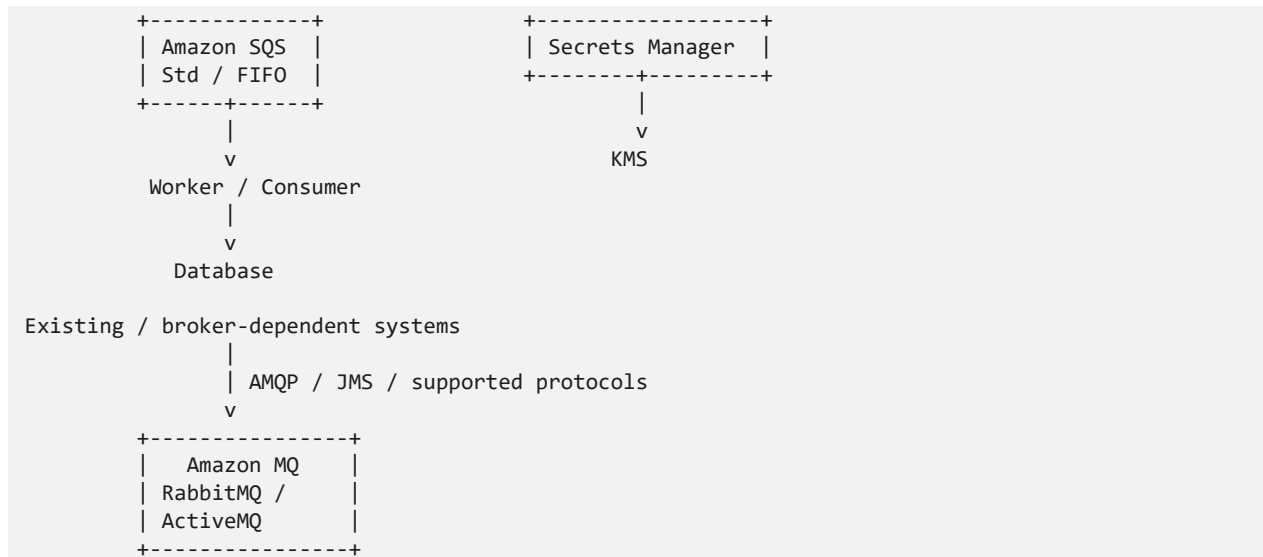
16.5 Monitoring and Auditing

- Use CloudTrail to audit Secrets Manager API activity.
- Monitor access patterns and investigate unexpected GetSecretValue calls.
- Use KMS audit information when customer-managed keys are involved.
- Never log the secret value itself.

17. Integrated Reference Architecture

17.1 Application Architecture





17.2 Why These Services Can Coexist

- SQS can handle asynchronous jobs between AWS-native application components.
- Amazon MQ can serve existing applications that require broker protocols and broker semantics.
- Secrets Manager can centralize credentials used by applications, workers and broker clients.
- KMS can provide centralized encryption-key control where customer-managed keys are required.
- CloudWatch and CloudTrail can provide operational monitoring and API auditing.

18. Security and Production Best Practices

Area	Best practice
IAM	Use least privilege. Prefer IAM roles for workloads.
SQS	Use DLQs, appropriate visibility timeout, long polling and idempotent consumers.
SQS FIFO	Choose MessageGroupId according to the business ordering requirement.
SQS payloads	Keep messages small; store large objects in S3 and send a reference.
Amazon MQ	Keep brokers private where possible and tightly control security groups.
Amazon MQ	Use TLS and broker authentication; verify supported protocols and versions.
Secrets Manager	Never hardcode credentials or commit them to source control.
Secrets Manager	Use caching and rotation where appropriate.
KMS	Use customer-managed keys only where additional key control is actually needed.
Monitoring	Use CloudWatch for operational metrics and CloudTrail for AWS API auditing.
Cost	Delete lab brokers and other chargeable resources immediately after testing.
Documentation	Record actual Region, versions, configuration and evidence instead of copying generic values.

19. Troubleshooting Runbook

19.1 SQS

Symptom	Likely checks	Action
Message appears repeatedly	Visibility timeout; consumer crash; missing DeleteMessage	Increase/adjust visibility timeout; fix consumer; delete after successful processing.
Queue backlog increases	Consumer capacity; errors; processing time	Inspect CloudWatch; scale workers; fix application failures.
AccessDenied	IAM policy; queue policy; KMS permissions	Review identity/resource policies and KMS permissions.
FIFO throttling	Regional FIFO quota; batching; message groups	Review current quota; use batching/high-throughput FIFO; distribute message groups.
Messages go to DLQ	Consumer errors; maxReceiveCount	Inspect failed message and consumer logs; fix root cause before redriving.

19.2 Amazon MQ

Symptom	Likely checks	Action
Cannot connect	Broker status; DNS; route; security group; port	Verify each network layer in order.
Authentication failed	Username/password; broker auth	Verify credentials without exposing them.
TLS error	Certificate trust; protocol/port; TLS settings	Use the broker's documented TLS endpoint and trusted certificates.
Protocol error	Client library vs engine mismatch	Use a client/protocol supported by the selected broker engine/version.
Unexpected cost	Broker still running; storage; data transfer	Stop/delete lab broker when finished; review billing.

19.3 Secrets Manager

Symptom	Likely checks	Action
AccessDeniedException	IAM role/policy; secret resource policy; KMS	Grant only required actions and correct resource/key permissions.
ResourceNotFoundException	Wrong Region/name/ARN	Verify Region and exact secret identifier.
Application slow / high API calls	GetSecretValue on every request	Implement appropriate client-side caching.
Unexpected secret exposure	Logs/tags/source code/CLI history	Rotate exposed credentials and remove the exposure path.

20. Validation and Evidence Checklist

Validation	Expected result	Status
SQS Standard queue	Queue created and visible	☐
SQS send	Message accepted	☐
SQS receive	Message retrieved	☐

Validation	Expected result	Status
Visibility timeout	Message temporarily hidden and later visible	☐
SQS delete	Message removed after successful processing	☐
FIFO ordering	Messages in same group preserve order	☐
FIFO deduplication	Duplicate test behaves according to configured deduplication	☐
DLQ	Repeatedly failing message reaches DLQ according to policy	☐
SQS security	IAM/encryption configuration documented	☐
Amazon MQ	Broker reaches RUNNING	☐
MQ networking	Client can reach broker using allowed path	☐
MQ messaging	Test message can be published/consumed	☐
Secrets Manager	Secret stored successfully	☐
Secret retrieval	Authorized identity can retrieve secret	☐
Least privilege	Only required secret actions granted	☐
KMS	Encryption configuration documented	☐
Rotation	Rotation configured/tested if included	☐
CloudTrail/CloudWatch	Required evidence captured	☐
Cleanup	Chargeable lab resources removed	☐

21. Cost and Cleanup Checklist

21.1 Cost Controls

- Use small lab configurations.
- Avoid unnecessary high-availability broker deployments during basic functional testing.
- Avoid leaving Amazon MQ brokers running after the lab.
- Keep SQS message traffic small during training.
- Avoid excessive Secrets Manager API calls; use caching in application tests.
- Check the AWS Billing console and Cost Explorer if available to your account.

21.2 Cleanup

128. Delete SQS test messages or delete the lab queues.
129. Delete the SQS DLQ after the source queue is no longer using it.
130. Delete the Amazon MQ broker after collecting all evidence. Confirm that deletion is intended because broker deletion is irreversible.
131. Terminate any EC2 test instance created only for the MQ lab.
132. Remove temporary security-group rules and lab-only IAM policies/roles if they are no longer needed.
133. Delete the Secrets Manager lab secret after documenting it. If recovery is configured, follow the documented deletion/recovery behavior and verify it does not remain unnecessarily.
134. Remove unused KMS keys only according to your organization's key-retention policy; do not schedule deletion of shared/production keys.
135. Recheck billing/cost usage after cleanup.

22. Interview / Viva Questions and Answers

Q: What is SQS?

A: A fully managed AWS message queuing service used to decouple application components and process messages asynchronously.

Q: Standard vs FIFO?

A: Standard is designed for high-scale workloads without ordering guarantees; FIFO provides ordered processing within message groups and deduplication capabilities.

Q: What is visibility timeout?

A: The period during which a received SQS message is hidden from other consumers while the current consumer processes it.

Q: Why use a DLQ?

A: To isolate repeatedly failing messages for troubleshooting instead of allowing endless retries in the main queue.

Q: What is long polling?

A: A receive operation that can wait for messages for up to 20 seconds, reducing empty receives.

Q: What is Amazon MQ?

A: A managed broker service for supported ActiveMQ and RabbitMQ workloads and protocols.

Q: Amazon MQ vs SQS?

A: SQS is an AWS-native queue; Amazon MQ provides managed traditional broker technologies and their supported protocols.

Q: Why use Amazon MQ?

A: It can support existing applications that depend on broker protocols, clients or semantics.

Q: What is Secrets Manager?

A: A managed service for securely storing, retrieving, controlling and rotating secrets such as credentials and API keys.

Q: Why not hardcode passwords?

A: Source code, configuration and logs can be copied or exposed. Secrets Manager centralizes secret access and lifecycle management.

Q: How does EC2 access a secret?

A: Through an IAM role attached to the instance, with `secretsmanager:GetSecretValue` permission scoped to the required secret.

Q: Why KMS?

A: To provide encryption-key management for protected data such as Secrets Manager secret values and supported SQS encryption configurations.

Q: Why cache secrets?

A: To reduce repeated API calls, latency and cost while still refreshing according to the application's rotation and security requirements.

23. Final Observations

- SQS provides a simple AWS-native way to decouple producers and consumers without managing a broker server.
- SQS message lifecycle behavior makes visibility timeout and `DeleteMessage` essential concepts for reliable consumers.
- FIFO queues should be selected when ordering and deduplication requirements are part of the application design.
- Dead-letter queues provide an operational isolation point for repeatedly failing messages.
- Amazon MQ is a broker service and therefore has stronger networking, broker-version and client/protocol considerations than a basic SQS queue.

- Amazon MQ is particularly relevant when an application already depends on ActiveMQ or RabbitMQ technologies.
- Secrets Manager separates credentials from application source code and provides controlled retrieval, encryption and rotation capabilities.
- IAM least privilege is a common security foundation across all three services.
- CloudWatch supports operational monitoring, while CloudTrail provides auditing of AWS API activity.
- Service quotas, supported versions, Region availability and pricing must be checked against current AWS documentation before production deployment.

24. Conclusion

The hands-on implementation establishes a clear understanding of three complementary AWS capabilities. Amazon SQS addresses asynchronous queue-based communication and decoupling; Amazon MQ provides managed traditional broker technologies for supported applications; and AWS Secrets Manager protects sensitive credentials and supports controlled lifecycle management. Together, these services can form part of a secure, resilient and maintainable cloud application architecture when each is selected according to the application's actual requirements.

Submission readiness: Before submitting, replace every blank evidence/status field, insert the actual screenshots, record the exact Region and versions used, add your observations from the real lab, and verify that no secret value is visible anywhere in the document.

25. Official AWS References

- **Amazon SQS Developer Guide:**
<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html>
- **Amazon SQS quotas:**
<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-quotas.html>
- **Amazon SQS FIFO quotas:**
<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/quotas-fifo.html>
- **Amazon SQS visibility timeout:**
<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html>
- **Amazon SQS dead-letter queues:**
<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html>
- **Amazon SQS encryption:**
<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-server-side-encryption.html>
- **Amazon SQS pricing:** <https://aws.amazon.com/sqs/pricing/>
- **Amazon MQ Developer Guide:** <https://docs.aws.amazon.com/amazon-mq/latest/developer-guide/>
- **Amazon MQ limits:** <https://docs.aws.amazon.com/amazon-mq/latest/developer-guide/amazon-mq-limits.html>
- **Amazon MQ pricing:** <https://aws.amazon.com/amazon-mq/pricing/>
- **AWS Secrets Manager User Guide:** <https://docs.aws.amazon.com/secretsmanager/latest/userguide/intro.html>
- **Secrets Manager best practices:** <https://docs.aws.amazon.com/secretsmanager/latest/userguide/best-practices.html>
- **Secrets Manager quotas:**
https://docs.aws.amazon.com/secretsmanager/latest/userguide/reference_limits.html
- **Secrets Manager rotation:** <https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html>

Secrets Manager pricing: <https://aws.amazon.com/secrets-manager/pricing/>

LAB 1 AWS SQS Standard Queue — Queue Creation

Objective

To create and configure an Amazon Simple Queue Service (SQS) Standard queue in the AWS Management Console and verify its basic configuration.

Purpose

The purpose of this task is to create an AWS-managed message queue that can be used to decouple application components and support asynchronous message processing.

The Standard queue will later be used to demonstrate the complete SQS message lifecycle:

Send → Queue → Receive → Process → Delete

Prerequisites

- AWS account with access to the AWS Management Console.
- Permission to create and manage Amazon SQS queues.
- AWS Region selected as **US East (N. Virginia) — us-east-1**.
- No existing queue with the same name, or an existing queue has been intentionally reviewed before reuse.
- No customer-managed KMS key is required for this basic lab.

Configuration

Configuration	Value
AWS Region	us-east-1
Queue Type	Standard
Queue Name	devops-demo-queue
Visibility Timeout	30 seconds
Message Retention	4 days
Delivery Delay	0 seconds
Receive Message Wait Time	20 seconds
Encryption	Enabled

Steps

1. Sign in to the AWS Management Console.
2. Select **US East (N. Virginia) — us-east-1**.
3. Search for and open **Amazon Simple Queue Service (SQS)**.
4. Check whether devops-demo-queue already exists.
5. Select **Create queue** if the queue does not already exist.
6. Select **Standard** as the queue type.
7. Enter devops-demo-queue as the queue name.
8. Configure the visibility timeout as **30 seconds**.
9. Configure message retention as **4 days**.

10. Configure delivery delay as **0 seconds**.
11. Configure the receive message wait time as **20 seconds**.
12. Keep SQS encryption enabled using the AWS-managed encryption option for this basic lab.
13. Review all configuration values.
14. Select **Create queue**.
15. Open the created queue and verify its configuration.

Screenshot

The screenshot displays the AWS Management Console interface for an Amazon SQS queue named "devops-demo-queue". At the top, there are buttons for "Edit", "Delete", "Purge", "Send and receive messages", and "Start DLQ redrive". Below these is the "Details" section, which includes a table with the following information:

Name	Type	ARN
devops-demo-queue	Standard	arn:aws:sqs:us-east-1:567487919764:devops-demo-queue
Encryption	URL	Dead-letter queue
Amazon SQS key (SSE-SQS)	https://sqs.us-east-1.amazonaws.com/567487919764/devops-demo-queue	-

Below the details section, there is a horizontal navigation bar with tabs for "Queue policies", "Monitoring", "SNS subscriptions", "Lambda triggers", "EventBridge Pipes", "Dead-letter queue", "Tagging", "Encryption", and "Dead-letter queue redrive tasks". The "Queue policies" tab is currently selected, showing the "Access policy" section. This section includes a text area with the following JSON policy:

```
{
  "Version": "2012-10-17",
  "Id": "__default_policy_id",
  "Statement": [
    {
      "Sid": "__owner_statement",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::567487919764:root"
      },
      "Action": "SQS:*",
      "Resource": "arn:aws:sqs:us-east-1:567487919764:devops-demo-queue"
    }
  ]
}
```

Observation

Record the actual result observed during the hands-on implementation. Do not use a generic observation if the configuration or AWS console behavior differs.

Example format:

The SQS Standard queue was successfully created in the us-east-1 Region. The configured queue settings were verified from the queue details page.

Conclusion

The Amazon SQS Standard queue was successfully created and its basic configuration was verified. The queue is now ready for the next stage of the lab, where messages will be sent, received, tested using the visibility timeout, and deleted.

LAB 2 AWS SQS — Sending a Message

Objective

To send a test message to the previously created Amazon SQS Standard queue and verify that the message was successfully accepted by the queue.

Purpose

The purpose of this task is to demonstrate how a producer/application sends a message to an SQS queue using the AWS Management Console.

This represents the first stage of the SQS message lifecycle:

Producer → SendMessage → SQS Queue

Prerequisites

- SQS Standard queue must already exist.
- Queue name: devops-demo-queue
- AWS Region: us-east-1
- User must have permission to send messages to the SQS queue.

Message Used

The following JSON message was used for testing:

```
{
  "orderId": "ORD1001",
  "customer": "Sunith",
  "amount": 2500,
  "status": "NEW"
}
```

Steps

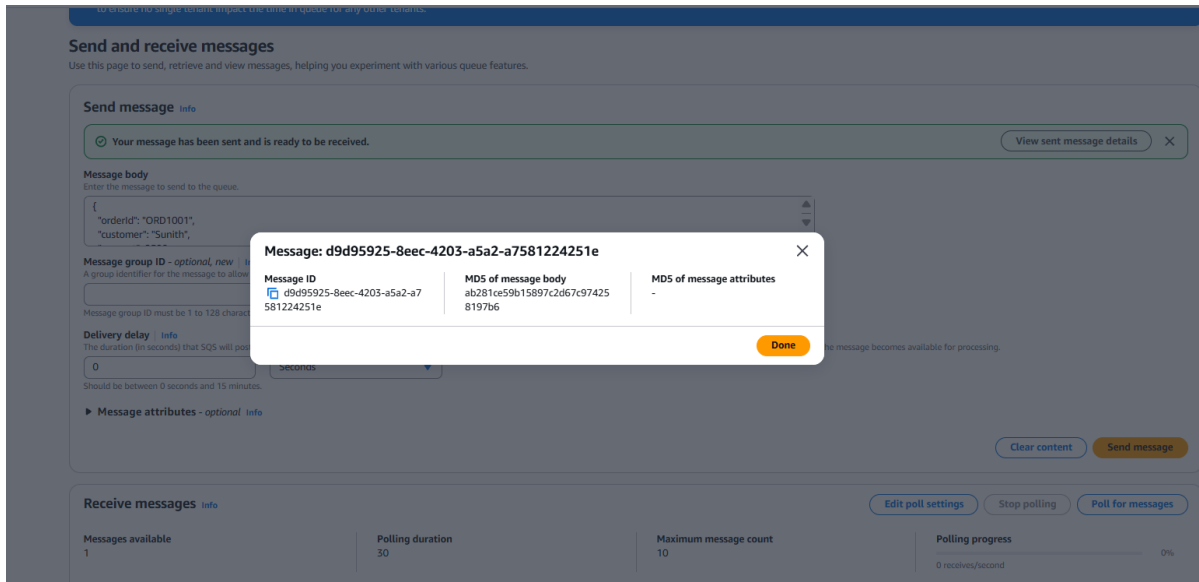
1. Open the AWS Management Console.
2. Open **Amazon Simple Queue Service (SQS)**.
3. Select **Queues**.
4. Open the devops-demo-queue Standard queue.
5. Select **Send and receive messages**.
6. Locate the message body field.
7. Enter the test JSON message.
8. Review the message content.
9. Select **Send message**.
10. Wait for the success confirmation.
11. Verify that the message was accepted by the SQS queue.
12. Record the result for the implementation document.

Verification

Verify that:

- The correct queue was selected.
- The message was sent successfully.
- The message contains the expected order information.
- No error or access-denied message was displayed.

Screenshot



Observation

Record the actual result observed during the lab.

Example:

The test order message was successfully sent to the devops-demo-queue SQS Standard queue.

Conclusion

A test JSON message was successfully sent to the SQS Standard queue. The queue is now ready for the receive operation, where the message will be retrieved and its message metadata and visibility behavior will be examined.

AWS SQS LAB 3 — Receive Message

Objective

To receive a message from an Amazon SQS Standard queue and verify the message details returned by Amazon SQS.

Purpose

The purpose of this lab is to understand how a consumer retrieves messages from an SQS queue. The message is received from the queue using the polling mechanism, while the message remains available for processing according to the queue's visibility timeout configuration.

Prerequisites

The following prerequisites were completed before starting this lab:

- AWS account is available.
- AWS Region is configured as us-east-1.
- SQS Standard queue devops-demo-queue was successfully created.
- The queue has a visibility timeout of 30 seconds.
- A test message was successfully sent to the queue.
- The message was not deleted before starting this lab.

Message Used

The following message was sent during the previous lab:

```
{  
  "orderId": "ORD1001",  
  "customer": "Sunith",  
  "amount": 2500,  
  "status": "NEW"  
}
```

Steps

Step 1 — Open Amazon SQS

1. Sign in to the AWS Management Console.
2. Open **Amazon Simple Queue Service (SQS)**.
3. Select the us-east-1 region.
4. Open the **Queues** section.

Step 2 — Open the Queue

Open the queue:

devops-demo-queue

Step 3 — Open Send and Receive Messages

Click:

Send and receive messages

Step 4 — Poll for Messages

In the receive section, click:

Poll for messages

Amazon SQS polls the queue and retrieves the available message.

Step 5 — Verify the Received Message

The received message was displayed successfully.

The following message information was observed:

- Message ID
- Message body
- Sent timestamp
- Receive count

The message body was verified against the message that was previously sent.

Step 6 — Do Not Delete the Message

The message was intentionally **not deleted**.

This is important because the next lab demonstrates the SQS Visibility Timeout mechanism.

Verification

The SQS polling operation completed successfully.

The message sent to:

devops-demo-queue

was successfully received.

The message body matched the previously sent test message.

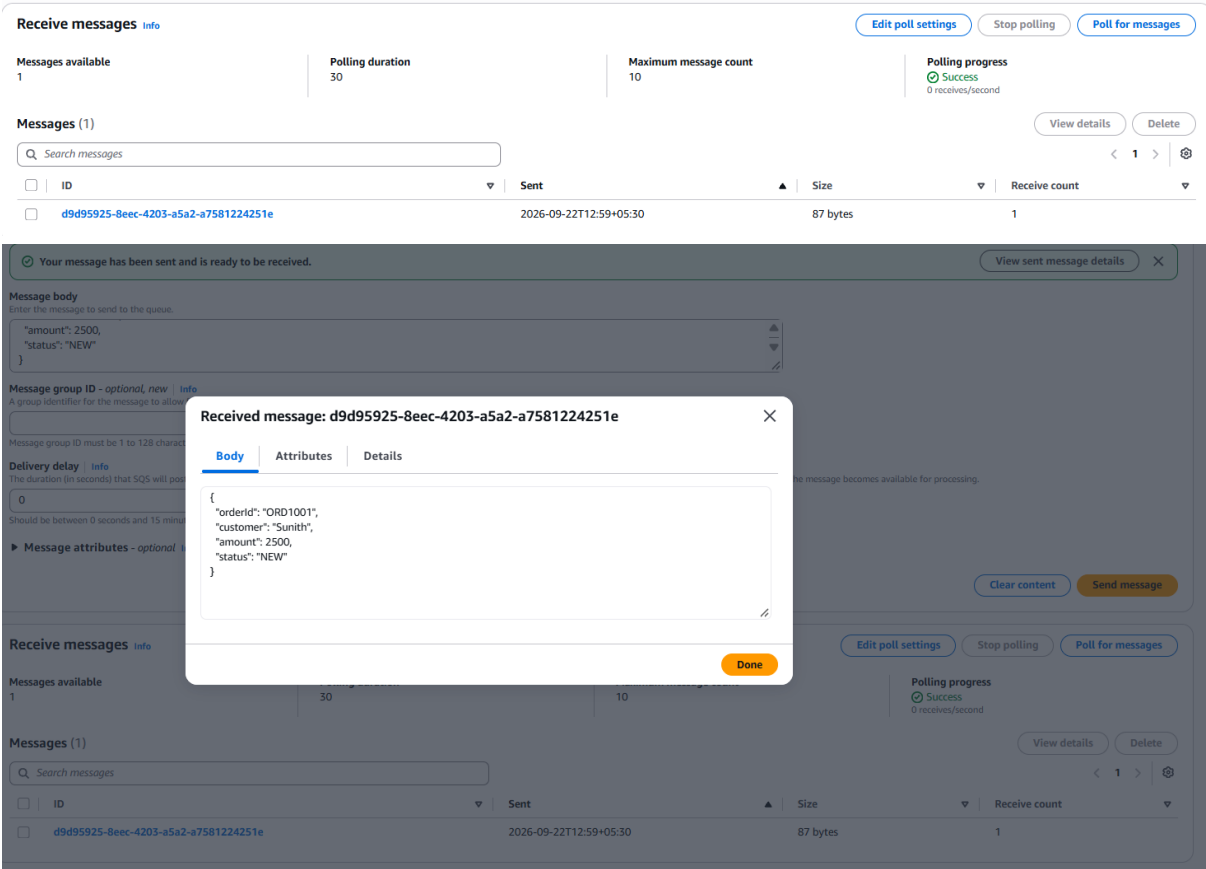
Observation

The message was successfully retrieved from the SQS queue using the **Poll for messages** operation.

The message was not automatically deleted after being received. This allows the consumer to process the message before explicitly deleting it.

The received message will be used in the next lab to demonstrate the Visibility Timeout behavior.

Screenshot:



Conclusion

The SQS Standard queue message was successfully received using the AWS SQS polling mechanism. The message details were verified, and the message was intentionally left undeleted for the next lab. This demonstrated the receive stage of the SQS message lifecycle.

AWS SQS LAB 4 — Understand Visibility Timeout

Objective

To understand and demonstrate how Amazon SQS Visibility Timeout temporarily hides a received message from subsequent receive requests.

Purpose

The purpose of this lab is to understand what happens when an SQS message is received but is not deleted by the consumer.

When a message is received, SQS temporarily makes the message invisible to other receive requests for the configured visibility timeout period. If the consumer does not delete the message during this period, the message becomes visible again and can be received again.

Prerequisites

The following conditions were completed before this lab:

- SQS Standard queue devops-demo-queue was created.
- Visibility timeout was configured as 30 seconds.
- Test message was successfully sent.
- The message was successfully received.
- The message was intentionally not deleted.

Queue Configuration

Queue Name:

devops-demo-queue

Queue Type:

Standard

Visibility Timeout:

30 seconds

Steps

Step 1 — Open the SQS Queue

Open:

Amazon SQS

→ Queues

→ devops-demo-queue

Step 2 — Open Send and Receive Messages

Click:

Send and receive messages

Step 3 — Poll for the Message

Click:

Poll for messages

The previously received message was displayed.

Step 4 — Do Not Delete the Message

The message was intentionally left undeleted.

Because the message was received but not deleted, it became temporarily invisible according to the configured visibility timeout.

Step 5 — Poll Again

After the visibility period expired, **Poll for messages** was used again.

The same message was displayed again.

Step 6 — Verify Receive Count

The received message showed:

Receive count = 2

The message body remained the same as the original message.

Verification

The following behavior was successfully observed:

First Receive

↓

Message becomes temporarily invisible

↓

Visibility Timeout expires

↓

Message becomes visible again

↓

Second Receive

↓

Receive count = 2

Observation

The same SQS message was successfully received again after the visibility timeout because it had not been deleted.

The receive count increased to 2, demonstrating that receiving a message does not automatically remove it from the queue.

Important Understanding

Receiving a message and deleting a message are two separate operations.

The normal processing lifecycle is:

Send

↓

Queue

↓

Receive

↓

Process

↓

Delete

If the consumer fails to delete the message before the visibility timeout expires, the message can become visible again.

Screenshot

The screenshot shows the AWS SQS console interface. At the top, there are buttons for 'Edit poll settings', 'Stop polling', and 'Poll for messages'. Below these, the 'Receive messages' section displays 'Messages available: 1', 'Polling duration: 30', and 'Maximum message count: 10'. On the right, the 'Polling progress' shows 'Success' with '0 receives/second'. A red arrow points to the 'Receive count' column in the message table, which is circled in red. The table shows a single message with ID 'd9d95925-8eec-4203-a5a2-a7581224251e', sent at '2026-09-22T12:59:05:30', with a size of '87 bytes', and a 'Receive count' of '2'. The 'Receive count' column header is also circled in red.

ID	Sent	Size	Receive count
d9d95925-8eec-4203-a5a2-a7581224251e	2026-09-22T12:59:05:30	87 bytes	2

Conclusion

The SQS Visibility Timeout mechanism was successfully demonstrated. The message was received, intentionally left undeleted, became visible again after the visibility timeout, and was subsequently received for the second time. The receive count increased to 2, confirming the behavior.

AWS SQS LAB 5 — Delete SQS Message

Objective

To delete a successfully received and processed message from an Amazon SQS Standard queue and verify that the deleted message is no longer available.

Purpose

The purpose of this lab is to understand the final stage of the SQS message lifecycle.

Receiving a message does not automatically remove it from the queue. After the consumer successfully processes the message, the consumer must explicitly delete the message.

The basic lifecycle demonstrated in this lab is:

Send ↓ Queue ↓ Receive ↓ Process ↓ Delete

Prerequisites

The following activities were completed before this lab:

- SQS Standard queue devops-demo-queue was successfully created.
- A test message was successfully sent.
- The message was successfully received.
- Visibility Timeout was successfully demonstrated.
- The message had previously been received multiple times because it was intentionally not deleted.
- The message was ready to be deleted.

Steps

Step 1 — Open the SQS Queue

1. Open the AWS Management Console.
2. Open **Amazon Simple Queue Service (SQS)**.
3. Select the us-east-1 region.
4. Open **Queues**.
5. Select:

devops-demo-queue

Step 2 — Open Send and Receive Messages

Click:

Send and receive messages

Step 3 — Poll for the Message

Click:

Poll for messages

The previously created message was retrieved.

The message body was verified as:

```
{  
  "orderId": "ORD1001",
```

```
"customer": "Sunith",  
"amount": 2500,  
"status": "NEW"  
}
```

Step 4 — Delete the Message

After confirming that the correct message was received, the message was deleted using the **Delete** operation.

The deletion was completed successfully.

Step 5 — Poll the Queue Again

After deletion, **Poll for messages** was selected again.

The previously deleted message did not appear.

This confirmed that the message was successfully removed from the queue.

Verification

The following verification was successfully completed:

Message received ↓ Message processed ↓ Message deleted ↓ Poll queue again ↓ Deleted message not returned

Observation

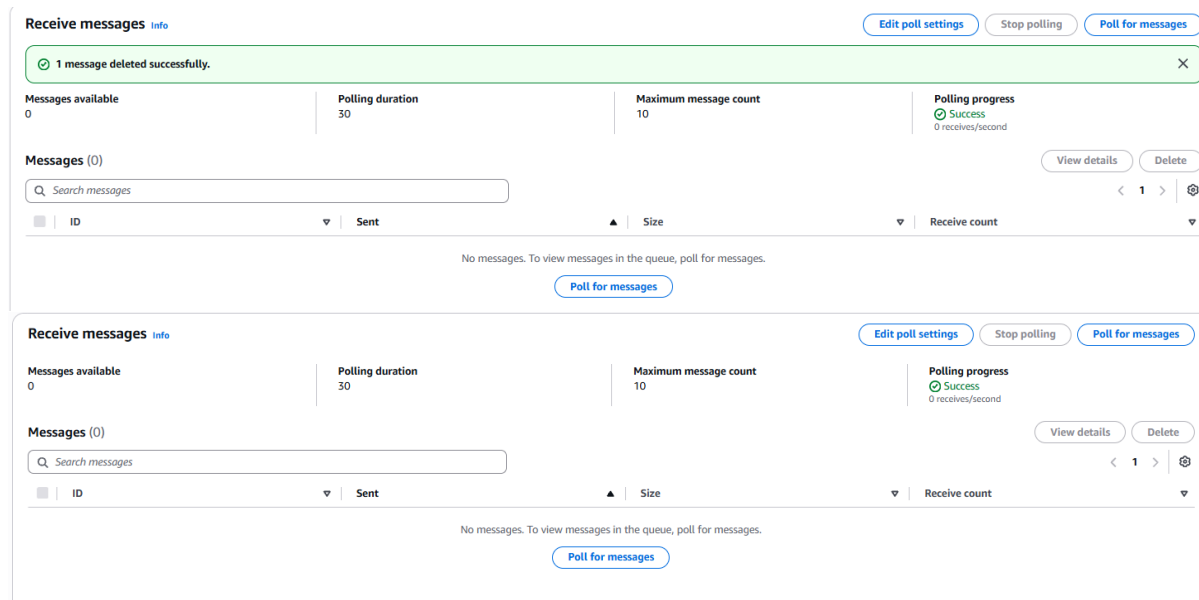
The message was not automatically deleted when it was received.

It remained available until the consumer explicitly performed the delete operation.

After deletion, polling the queue again did not return the deleted message.

This demonstrated the importance of deleting an SQS message after successful processing.

Screenshot:



Complete Standard SQS Message Lifecycle

The hands-on exercises demonstrated the complete basic lifecycle:

Produce -> Send Message → | SQS Queue | Receive → Consumer | Process → Delete
→ Message Removed

Conclusion

The SQS message was successfully deleted after processing. Polling the queue again confirmed that the deleted message was no longer available.

The complete Standard SQS message lifecycle — **Send → Queue → Receive → Process → Delete** — was successfully demonstrated.

LAB 6 — Amazon SQS FIFO Queue

1. Objective

To create and configure an Amazon Simple Queue Service (SQS) FIFO queue, enable content-based deduplication, send multiple messages using the same Message Group ID, and verify that the messages are received in the same order in which they were sent.

2. Purpose

The purpose of this lab is to understand the behavior of an SQS FIFO queue and demonstrate ordered message processing.

FIFO means:

First In, First Out

Unlike a Standard queue, a FIFO queue is designed for workloads where message ordering is important. FIFO queues also provide deduplication capabilities.

3. AWS Service Used

Service: Amazon Simple Queue Service (Amazon SQS)

Region: US East (N. Virginia) — us-east-1

Queue Type: FIFO

4. Queue Configuration

The following configuration was used:

Configuration	Value
Queue Name	devops-demo-orders.fifo
Queue Type	FIFO
FIFO Suffix	.fifo
Content-Based Deduplication Enabled	
Message Group ID	customer-1
Messages	Order 1, Order 2, Order 3

The .fifo suffix is required for FIFO queue names.

Step 1 — Open Amazon SQS

1. Sign in to the AWS Management Console.
 2. Select the required AWS Region.
 3. Search for **SQS**.
 4. Open **Amazon Simple Queue Service**.
-

Step 2 — Create the FIFO Queue

Create a new queue with the following configuration:

Queue Name:

devops-demo-orders.fifo

Select:

FIFO

The queue name must contain: `.fifo`

After configuring the queue, create it.

Verification

Confirm that the queue appears in the SQS queue list as:

devops-demo-orders.fifo

and that its type is:

FIFO

Step 3 — Enable Content-Based Deduplication

During queue configuration, enable:

Content-based deduplication

This configuration allows SQS to use message content when performing deduplication for the FIFO queue.

Step 4 — Send FIFO Messages

Open:devops-demo-orders.fifo

Select:

Send and receive messages

Send the following three messages.

Message 1

Message Body: Order 1

MessageGroupId: customer-1

Send the message successfully.

Message 2

Message Body: Order 2

MessageGroupId: customer-1

Send the message successfully.

Message 3

Message Body: Order 3
MessageGroupId: customer-1
Send the message successfully.
All three messages were successfully sent.

5. Receive and Verify FIFO Ordering

After sending the messages, use:
Receive messages
and:
Poll for messages
The messages were received and verified in the following order:
Order 1 ↓ Order 2 ↓ Order 3
All messages used:
MessageGroupId = customer-1
Each received message was successfully deleted after verification.

6. Important Observation

All three messages belonged to the same:
MessageGroupId = customer-1
The lab demonstrated ordered processing within that message group. The source material explains that messages within a Message Group maintain ordering, while multiple groups can be used when parallelism is required.

7. FIFO vs Standard Queue

Feature	Standard Queue	FIFO Queue
Ordering	Not guaranteed	Ordered within message groups
Queue naming	Normal queue name	Must end with .fifo
Deduplication	Not the FIFO content-based mechanism	Supports FIFO deduplication
Message Groups	Not applicable	Supported
Typical use	Very high-scale asynchronous workloads	Workloads where ordering matters

The source material identifies FIFO queues as appropriate when ordering is important, such as sequential order processing and transactions.

8.Screenshot:

Receive messages [Info](#)

Messages available3

Polling duration30

Maximum message count10

Polling progress

0 receives/second

53%

View details

Delete

Search messages

☐	ID	Sent	Size	Receive count
☐	f3da7173-ccbe-4ea5-b30e-24c969aa05fc	2026-09-22T14:42+05:30	8 bytes	1
☐	a61fadd0-1d19-4a25-8303-362cbd2d517d	2026-09-22T14:42+05:30	7 bytes	1
☐	1e7f8754-6a49-4cfd-a7c2-ff91d6393716	2026-09-22T14:42+05:30	7 bytes	1

Received message: f3da7173-ccbe-4ea5-b30e-24c969aa05fc

Body

Attributes

Details

order 1

Done

Received message: a61fadd0-1d19-4a25-8303-362cbd2d517d

Body

Attributes

Details

order 2

Done

Received message: 1e7f8754-6a49-4cfd-a7c2-ff91d6393716

Body

Attributes

Details

order 3

Done

9. Troubleshooting Notes

Problem: FIFO queue name rejected

Verify that the queue name ends with:

.fifo

Correct:

devops-demo-orders.fifo

Incorrect:

devops-demo-orders

Problem: Messages are not appearing immediately

Check whether the message has already been received and is currently invisible because of the visibility timeout.

Also verify that the message was not already deleted.

Problem: FIFO ordering is not as expected

Verify that the messages being compared belong to the same:

MessageGroupId

For this lab, the required value was:

customer-1

10. Key Interview Points

What is FIFO?

FIFO stands for:

First In, First Out

It is an SQS queue type designed for ordered message processing.

Why is .fifo important?

FIFO queue names must end with:

.fifo

What is MessageGroupId?

MessageGroupId identifies a group of related messages for FIFO processing.

Messages within the same group maintain their ordering.

What was demonstrated in this lab?

Order 1 ↓ Order 2 ↓ Order 3

The messages were received in the same sequence in which they were sent.

Conclusion

The Amazon SQS FIFO queue hands-on implementation was completed successfully.

A FIFO queue named:

devops-demo-orders.fifo

was created with content-based deduplication enabled. Three messages — Order 1, Order 2, and Order 3 — were sent using the same MessageGroupId, customer-1.

The messages were subsequently received and verified in the expected order:

Order 1 → Order 2 → Order 3

The messages were then deleted successfully.

Therefore, the lab successfully demonstrated the basic implementation and ordered-processing behavior of an Amazon SQS FIFO queue.

LAB 7 AWS SQS — Dead-Letter Queue (DLQ) Implementation

1. Objective

To create and configure an Amazon Simple Queue Service (SQS) Dead-Letter Queue (DLQ), associate it with the main SQS queue, configure the maximum receive count, and verify that a repeatedly received message is automatically moved from the source queue to the DLQ.

2. Purpose

A Dead-Letter Queue is used to isolate messages that cannot be successfully processed after a configured number of receive attempts.

In a production environment, a message may repeatedly fail because of:

- Invalid message data
- Application processing errors
- Database connectivity problems
- Downstream service failures
- Invalid business logic
- Temporary or permanent application failures

Instead of allowing such a message to repeatedly return to the main queue, SQS can move it to a DLQ after the configured maximum receive count is reached.

The DLQ allows administrators and developers to:

- Identify failed messages
- Investigate processing failures
- Monitor problematic messages
- Prevent continuously failing messages from affecting normal processing
- Reprocess or remove failed messages when appropriate

3. AWS Resources Used

Resource	Configuration
AWS Service	Amazon SQS
AWS Region	US East (N. Virginia) — us-east-1
Source Queue	devops-demo-queue
Source Queue Type	Standard
Dead-Letter Queue	devops-demo-dlq
DLQ Type	Standard
Maximum Receive Count	3
Visibility Timeout	30 seconds

Resource	Configuration
Test Message ID	57683155-6aea-4f08-802b-ca765bc39085

4. Prerequisites

Before configuring the DLQ, the following requirements were verified:

1. AWS account access was available.
 2. AWS Region was set to us-east-1.
 3. The source SQS queue devops-demo-queue already existed.
 4. The source queue was a Standard queue.
 5. A separate Standard queue was created for use as the DLQ.
 6. The Dead-Letter Queue redrive configuration was enabled on the source queue.
 7. Maximum receive count was configured as 3.
 8. The source queue visibility timeout was 30 seconds.
-

5. Implementation Steps

Step 1 — Open Amazon SQS

1. Sign in to the AWS Management Console.
2. Select the AWS Region:

US East (N. Virginia) — us-east-1

3. Search for **SQS**.
 4. Open **Amazon Simple Queue Service**.
 5. Select **Queues**.
-

Step 2 — Verify the Source Queue

The existing source queue was verified:

devops-demo-queue

The queue type was:

Standard

This queue was used as the main/source queue for the DLQ test.

Step 3 — Create the Dead-Letter Queue

A separate SQS queue was created with the name:

devops-demo-dlq

Because the source queue was a Standard queue, the DLQ was configured as a Standard queue as well.

The DLQ was successfully created.

Step 4 — Configure the Dead-Letter Queue on the Source Queue

The source queue was opened:

devops-demo-queue

The Dead-Letter Queue configuration was enabled.

The DLQ selected was:

devops-demo-dlq

The maximum receive count was configured as:

3

The configuration was then saved successfully.

AWS displayed confirmation that the configuration for:

devops-demo-queue

was saved successfully.

Step 5 — Verify DLQ Configuration

The following configuration was verified:

Source Queue:

devops-demo-queue

Dead-Letter Queue:

devops-demo-dlq

Dead-Letter Queue:

Enabled

Maximum Receive Count:

3

The configuration was successfully saved.

6. DLQ Testing

After configuring the DLQ, an SQS test message was used to verify the redrive behavior.

The message ID observed during testing was:

57683155-6aea-4f08-802b-ca765bc39085

The message was intentionally received without being deleted.

Step 6 — First Receive

The test message was received from:

devops-demo-queue

The message was not deleted.

Because the visibility timeout was:

30 seconds

the message temporarily became invisible.

Step 7 — Subsequent Receive

After the visibility timeout expired, the queue was polled again.

The same message became available for receiving again.

The receive count increased.

The message was again intentionally left undeleted.

Step 8 — Reach the Maximum Receive Count

The message was repeatedly received without being deleted.

The receive count reached the configured threshold:

Maximum Receive Count = 3

The message was then removed from the source queue as part of the DLQ redrive behavior.

7. Final DLQ Verification

After the message disappeared from:

devops-demo-queue

the DLQ was opened:

Amazon SQS

→ Queues

→ devops-demo-dlq

→ Send and receive messages

The **Poll for messages** operation was performed.

The previously failing test message appeared in:

devops-demo-dlq

This confirmed that the DLQ redrive configuration was functioning correctly.

8. Actual Observation

During the hands-on test, the message with ID:

57683155-6aea-4f08-802b-ca765bc39085

was repeatedly received from devops-demo-queue without being deleted.

The message initially showed a receive count of 2. After continuing the test, the message disappeared from the source queue.

The devops-demo-dlq queue was then opened and **Poll for messages** was selected. The test message appeared in the DLQ.

This confirmed that the configured Dead-Letter Queue redrive policy was working successfully.

9. Important Concepts Learned

Dead-Letter Queue

A DLQ is a separate SQS queue used to store messages that could not be successfully processed after the configured number of receive attempts.

Maximum Receive Count

The maximum receive count determines how many times a message can be received from the source queue before SQS redrives it to the DLQ according to the configured redrive policy.

In this lab:

Maximum Receive Count = 3

Visibility Timeout

The visibility timeout temporarily hides a message after it is received.

In this lab:

Visibility Timeout = 30 seconds

If the consumer does not delete the message during the visibility period, it can become visible again.

Message Deletion

Receiving a message does not automatically delete it.

A successfully processed message should normally be deleted by the consumer.

In this lab, the message was intentionally not deleted so that DLQ behavior could be demonstrated.

10. Screenshots

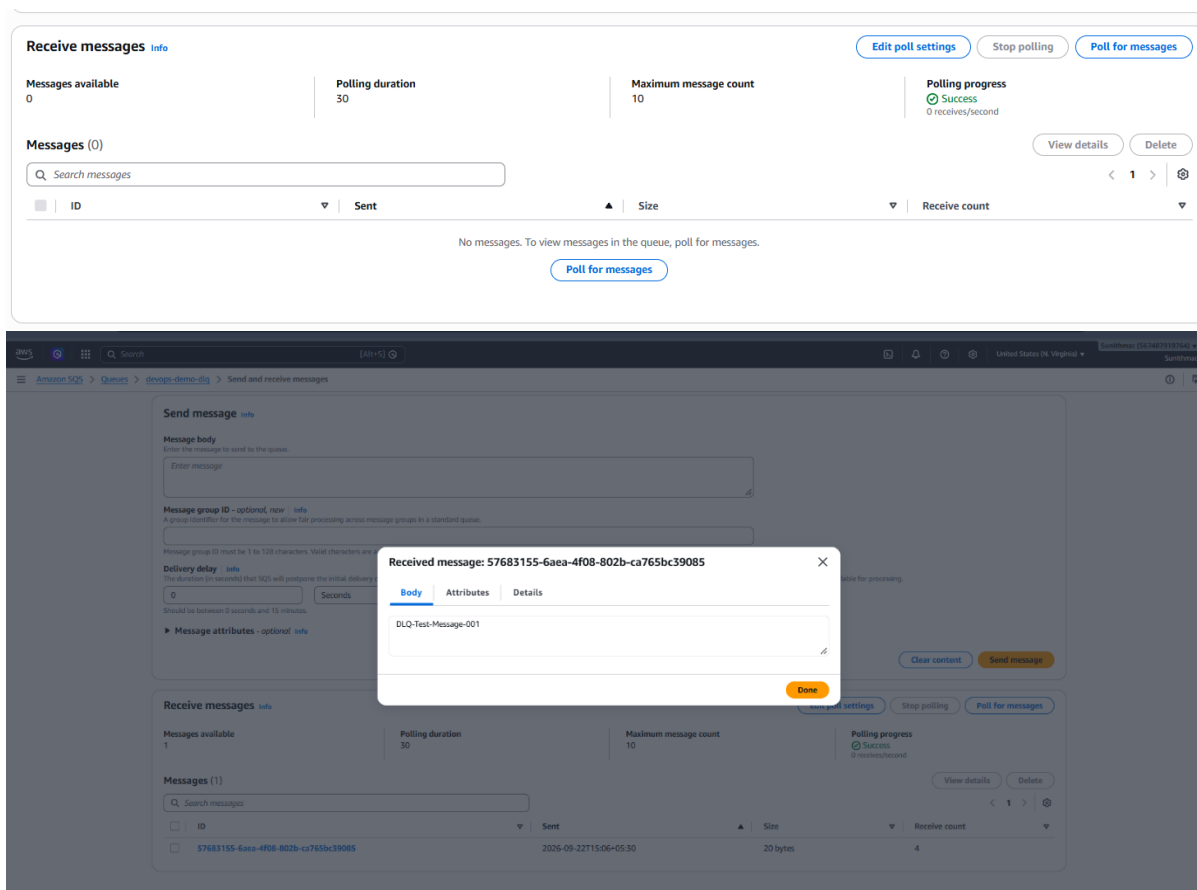
The screenshot displays the AWS Management Console interface for a Dead-Letter Queue (DLQ) named 'devops-demo-dlq'. At the top, a green notification bar states: 'Queue devops-demo-dlq created successfully. You can now send and receive messages.' Below this, the queue's details are shown in a table:

Details	
Name devops-demo-dlq	Type Standard
Encryption Amazon SQS key (SSE-SQS)	URL https://sqs.us-east-1.amazonaws.com/567487919764/devops-demo-dlq
ARN arn:aws:sqs:us-east-1:567487919764:devops-demo-dlq	
Dead-letter queue -	

Below the details table, there are tabs for 'Queue policies', 'Monitoring', 'SNS subscriptions', 'Lambda triggers', 'EventBridge Pipes', 'Dead-letter queue', 'Tagging', 'Encryption', and 'Dead-letter queue redrive tasks'. The 'Access policy' tab is currently selected, showing a JSON policy document:

```
{
  "Version": "2012-10-17",
  "Id": "__default_policy_ID",
  "Statement": [
    {
      "Sid": "__owner_statement",
      "Effect": "Allow",
      "Principal": "*"
    }
  ]
}
```

Overlaid on the bottom of the console is a modal window titled 'Received message: 57683155-6aea-4f08-802b-ca765bc39085'. The modal has three tabs: 'Body', 'Attributes', and 'Details'. The 'Body' tab is active, showing the message content: 'DLQ-Test-Message-001'. At the bottom right of the modal is a yellow 'Done' button.



11. Troubleshooting

Problem 1 — Message does not move to the DLQ

Check:

1. DLQ is attached to the source queue.
2. DLQ configuration is enabled.
3. Maximum receive count is configured correctly.
4. Message is not being deleted.
5. Visibility timeout has expired between receive attempts.
6. The correct queue is being polled.

Problem 2 — Message disappears from the source queue but is not immediately visible in the DLQ

Check the DLQ directly and select:

Poll for messages

SQS console message availability can require polling before the message is displayed.

Problem 3 — Message keeps returning to the source queue

Check:

Maximum Receive Count

and verify that the DLQ redrive policy was actually saved.

Also verify that the message is not being successfully deleted by another consumer.

Problem 4 — DLQ cannot be selected

Verify that:

- The DLQ exists.
 - The DLQ is in the same AWS Region.
 - The queue types are compatible.
 - You have the required SQS permissions.
-

12. Security and Operational Considerations

In a production environment:

- DLQs should be monitored.
 - Failed messages should be investigated rather than blindly deleted.
 - Access to DLQ messages should be restricted using appropriate IAM permissions.
 - Sensitive message data should be protected using SQS encryption.
 - CloudWatch monitoring and alarms can be used to detect messages accumulating in a DLQ.
 - DLQ retention should be selected according to the application's troubleshooting and recovery requirements.
 - Redrive/reprocessing should be performed carefully to prevent repeatedly failing messages from returning indefinitely to the main processing path.
-

13. Conclusion

The Amazon SQS Dead-Letter Queue mechanism was successfully implemented and tested.

The hands-on exercise demonstrated how a repeatedly received and unprocessed message can be isolated from the main queue and moved to a dedicated Dead-Letter Queue after the configured maximum receive count is reached.

This lab provided practical understanding of:

- SQS Dead-Letter Queues
- Redrive policies
- Maximum receive count
- Visibility timeout
- Message receive attempts
- Message deletion
- Failed-message isolation
- DLQ verification and troubleshooting

The SQS DLQ task is therefore **successfully completed**.

LAB 8 — AMAZON SQS SECURITY: IAM, ENCRYPTION AND AWS KMS

1. Objective

The objective of this lab is to secure an Amazon SQS queue using:

- AWS Identity and Access Management (IAM)
- IAM least-privilege permissions
- AWS Key Management Service (AWS KMS)
- Customer-managed KMS key
- Server-Side Encryption with AWS KMS (SSE-KMS)
- IAM role-based access
- Secure SQS message operations

The lab demonstrates how an application or user can securely interact with an encrypted SQS queue without granting unnecessary permissions.

2. Purpose

The purpose of this lab is to understand and implement security controls for Amazon SQS.

The implementation provides:

1. Encryption of SQS messages using a customer-managed AWS KMS key.
 2. Least-privilege permissions for SQS operations.
 3. Restricted KMS permissions.
 4. Role-based access instead of using the AWS root account for application operations.
 5. Resource-level restriction to a specific SQS queue.
 6. Secure testing of SQS operations using an assumed IAM role.
-

3. AWS Environment

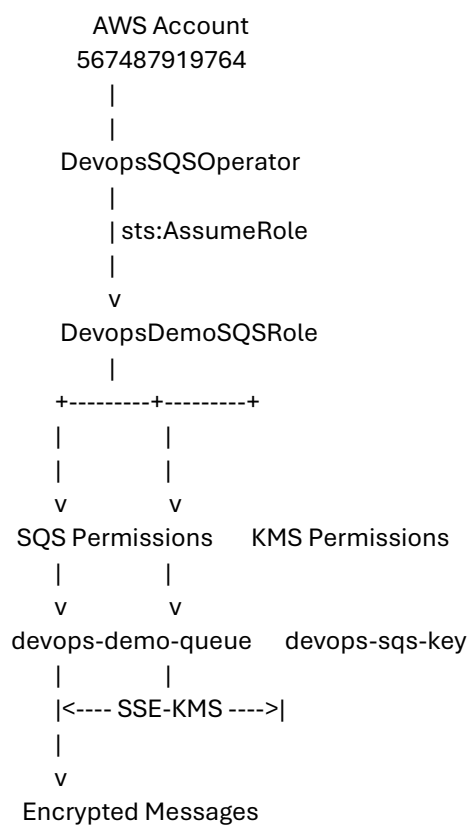
Configuration Value

AWS Account ID 567487919764	
AWS Region	us-east-1
Region Name	US East (N. Virginia)
SQS Queue	devops-demo-queue
Queue Type	Standard
Queue ARN	arn:aws:sqs:us-east-1:567487919764:devops-demo-queue
DLQ	devops-demo-dlq
Encryption	SSE-KMS

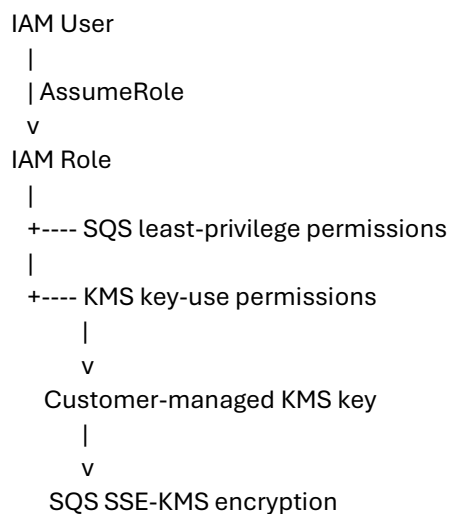
Configuration	Value
KMS Key Alias	devops-sqs-key
IAM Role	DevopsDemoSQSRole
IAM User	DevopsSQSOperator

4. Architecture

The implemented security architecture is:



Security flow



5. Existing SQS Queue

The existing SQS queue used for this lab was:

Queue Name:

devops-demo-queue

Queue ARN:

arn:aws:sqs:us-east-1:567487919764:devops-demo-queue

Queue Type:

Standard

The queue already had a Dead-Letter Queue configured:

devops-demo-dlq

The queue was originally configured with SQS-managed encryption:

Amazon SQS key (SSE-SQS)

For this security lab, the encryption was changed to a customer-managed KMS key.

6. Create Customer-Managed KMS Key

A dedicated customer-managed KMS key was created for the SQS queue.

KMS Key Details

Property	Value
Alias	devops-sqs-key
Key ID	dc350503-4ed1-4857-aa04-8462d3200eca
Key ARN	arn:aws:kms:us-east-1:567487919764:key/dc350503-4ed1-4857-aa04-8462d3200eca
Status	Enabled
Key Type	Symmetric
Key Usage	Encrypt and decrypt
Region	us-east-1

Description Customer-managed KMS key for devops-demo-queue SQS encryption

The KMS key was created specifically for the SQS security implementation.

7. Configure SQS SSE-KMS Encryption

The SQS queue was configured to use the customer-managed KMS key.

Navigation

AWS Console:

AWS Console

→ Amazon SQS

→ Queues

→ devops-demo-queue

→ Encryption

The encryption configuration was changed from:

Amazon SQS key (SSE-SQS)

to:

AWS Key Management Service key (SSE-KMS)

The selected KMS key was:

devops-sqs-key

After saving the configuration, the queue displayed:

Encryption:

AWS Key Management Service key (SSE-KMS)

This means messages stored in the queue are protected using the configured customer-managed KMS key.

8. Create SQS Least-Privilege IAM Policy

A custom IAM policy was created:

DevopsDemoSQSLeastPrivilege

Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "SQSAccessToDevopsDemoQueue",
      "Effect": "Allow",
      "Action": [
        "sqs:GetQueueUrl",
        "sqs:GetQueueAttributes",
        "sqs:SendMessage",
        "sqs:ReceiveMessage",
        "sqs:DeleteMessage"
      ],
      "Resource": "arn:aws:sqs:us-east-1:567487919764:devops-demo-queue"
    }
  ]
}
```

Explanation

The policy permits only the required SQS operations:

Permission	Purpose
sqs:GetQueueUrl	Obtain the queue URL
sqs:GetQueueAttributes	Read queue attributes
sqs:SendMessage	Send messages
sqs:ReceiveMessage	Receive messages

Permission	Purpose
sqs:DeleteMessage	Delete successfully processed messages

The policy is restricted to:

arn:aws:sqs:us-east-1:567487919764:devops-demo-queue

Therefore, the role does not receive unrestricted access to all SQS queues.

9. Create KMS Use-Key IAM Policy

A second custom policy was created:

DevopsDemoSQS-KMS-UseKey

Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "UseDevopsSQSKMSKey",
      "Effect": "Allow",
      "Action": [
        "kms:Decrypt",
        "kms:GenerateDataKey"
      ],
      "Resource": "arn:aws:kms:us-east-1:567487919764:key/dc350503-4ed1-4857-aa04-8462d3200eca"
    }
  ]
}
```

Explanation

The policy allows the role to use the specific KMS key for the cryptographic operations required by the encrypted SQS workload.

The resource is restricted to:

devops-sqs-key

rather than allowing access to every KMS key.

10. Create IAM Role

A dedicated IAM role was created:

DevopsDemoSQSRole

The purpose of this role is to provide controlled access to:

- devops-demo-queue
- devops-sqs-key

The following policies were attached to the role:

DevopsDemoSQSLeastPrivilege

DevopsDemoSQS-KMS-UseKey

11. Configure IAM Role Trust Policy

The role trust policy was configured as:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::567487919764:root"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

This allows principals within the AWS account to assume the role when they have the appropriate sts:AssumeRole permission.

12. Configure KMS Key User

The KMS key was configured with:

Key users (1)

DevopsDemoSQSRole

Therefore, DevopsDemoSQSRole is explicitly configured as a user of the customer-managed KMS key.

The KMS key policy also contains:

```
{
  "Sid": "Enable IAM User Permissions",
  "Effect": "Allow",
  "Principal": {
    "AWS": "arn:aws:iam::567487919764:root"
  },
  "Action": "kms:*",
  "Resource": "*"
}
```

This allows IAM permissions in the account to participate in KMS authorization.

The KMS key policy also explicitly permits DevopsDemoSQSRole to use the key.

13. Create IAM User for CLI Testing

Initially, the AWS CLI was authenticated using root credentials.

The root account cannot directly assume an IAM role through the AssumeRole operation.

Therefore, a dedicated IAM user was created:

DevopsSQSOperator

This user was created specifically for the lab's role-assumption test.

14. Create AssumeRole Policy

A custom policy was created:

DevopsSQSAssumeRolePolicy

Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAssumeDevopsDemoSQSRole",
      "Effect": "Allow",
      "Action": "sts:AssumeRole",
      "Resource": "arn:aws:iam::567487919764:role/DevopsDemoSQSRole"
    }
  ]
}
```

This allows the user to assume only:

DevopsDemoSQSRole

It does not provide unrestricted role-assumption permissions.

15. Configure Git Bash

The AWS CLI was configured from Git Bash.

The invalid old environment credentials were cleared:

```
unset AWS_ACCESS_KEY_ID
```

```
unset AWS_SECRET_ACCESS_KEY
```

```
unset AWS_SESSION_TOKEN
```

Then the AWS CLI was configured using:

```
aws configure
```

Configuration:

Region:

us-east-1

Output:

json

The credentials used belong to:

DevopsSQSOperator

Credentials were not exposed or shared.

16. Verify IAM Identity

The initial root-based CLI configuration caused:

Roles may not be assumed by root accounts.

After configuring the non-root IAM user and assuming the role, the following command was executed:

`aws sts get-caller-identity`

Successful result:

```
{
  "UserId": "AROAYIIHKO2KOESCFFDXM:SQS-Security-Lab",
  "Account": "567487919764",
  "Arn": "arn:aws:sts::567487919764:assumed-role/DevopsDemoSQSRole/SQS-Security-Lab"
}
```

Verification

This confirmed that the Git Bash session was operating as:

DevopsDemoSQSRole

and not as the root account.

17. SQS SendMessage Security Test

The first functional security test was performed using the assumed role.

Command:

```
aws sqs send-message \
  --queue-url "https://sqs.us-east-1.amazonaws.com/567487919764/devops-demo-queue" \
  --message-body '{"orderId":"LAB8-001","customer":"Sunith","amount":1500,"status":"SECURITY-TEST"}' \
  --region us-east-1
```

Result

The operation was successful.

Response:

```
{
  "MD5OfMessageBody": "8154b1446a8aaf2fab232d054f38f0b1",
  "MessageId": "ae1c1e4a-5c95-44d8-9e3e-b72760c94889"
}
```

Observation

The successful result proves that the assumed role had sufficient permissions to send a message to:

devops-demo-queue

while the queue was protected using:

SSE-KMS

This validates the configured relationship between:

IAM Role



SQS permissions



SQS Queue



Customer-managed KMS key

Test 3 — DeleteMessage

After receiving the message, delete it using:

```
aws sqs delete-message \  
--queue-url "https://sqs.us-east-1.amazonaws.com/567487919764/devops-demo-queue" \  
--receipt-handle "<RECEIPT_HANDLE>" \  
--region us-east-1
```

A successful delete verifies the role has:

sqs:DeleteMessage

Test 4 — Least-Privilege AccessDenied Test

After normal testing is complete, test an operation that was deliberately not included in the IAM policy.

For example:

```
aws sqs purge-queue \  
--queue-url "https://sqs.us-east-1.amazonaws.com/567487919764/devops-demo-queue" \  
--region us-east-1
```

The expected result is:

AccessDenied

This demonstrates that the role does not have unrestricted SQS permissions.

IMPORTANT:

Do this only after the normal message testing is finished because PurgeQueue removes all messages from the queue.

18. Security Concepts Demonstrated

IAM Least Privilege

Instead of granting:

AmazonSQSFullAccess

the custom policy grants only the required actions.

This reduces unnecessary permissions.

IAM Role

The application/test identity assumes:

DevopsDemoSQSRole

instead of performing application operations with the root account.

Customer-Managed KMS Key

A dedicated KMS key was created:

devops-sqs-key

This gives greater control over key permissions and key usage.

SSE-KMS

The SQS queue uses:

AWS KMS

for server-side encryption.

Resource-Level Restriction

The SQS IAM policy is restricted to:

arn:aws:sqs:us-east-1:567487919764:devops-demo-queue

The KMS IAM policy is restricted to:

arn:aws:kms:us-east-1:567487919764:key/dc350503-4ed1-4857-aa04-8462d3200eca

This prevents the role from automatically receiving access to unrelated resources.

19. Troubleshooting Performed

Problem 1 — Root account could not assume role

Error:

Roles may not be assumed by root accounts.

Cause

The AWS CLI was authenticated using root credentials.

Resolution

A dedicated IAM user was created:

DevopsSQSOperator

The user was given permission to assume:

DevopsDemoSQSRole

Problem 2 — Invalid AWS CLI security token

Error:

InvalidClientTokenId

The security token included in the request is invalid

Cause

Git Bash was using invalid or outdated credential information.

Resolution

The environment variables were cleared:

unset AWS_ACCESS_KEY_ID

unset AWS_SECRET_ACCESS_KEY

unset AWS_SESSION_TOKEN

Then the correct IAM user credentials were configured with:

```
aws configure
```

The identity was subsequently verified using:

```
aws sts get-caller-identity
```

20. Important Security Precautions

Never share:

- AWS Secret Access Keys
- AWS Session Tokens
- IAM passwords
- Private SSH keys
- Temporary role credentials

Safe documentation values include:

- AWS Account ID
- Queue ARN
- KMS Key ARN
- IAM role name
- IAM policy JSON
- Message ID

Credentials should always be kept private.

21. Recommended Screenshots

Policy DevopsDemoSQS-KMS-UseKey created.

View policy

DevopsDemoSQS-KMS-UseKey

info

Edit

Delete

Allows required KMS operations for the encrypted devops-demo-queue.

Policy details

Type	Creation time	Edited time	ARN
Customer managed	September 22, 2026, 16:04 (UTC+05:30)	September 22, 2026, 16:04 (UTC+05:30)	arn:aws:iam::567487919764:policy/DevopsDemoSQS-KMS-UseKey

Permissions

Entities attached

Tags

Policy versions (1)

Last Accessed

Permissions defined in this policy

info

Copy

Edit

Summary

JSON

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "UseDevopsSQSKMSKey",
6       "Effect": "Allow",
7       "Action": [
8         "kms:Decrypt",
9         "kms:GenerateDataKey"
10      ],
11       "Resource": "arn:aws:kms:us-east-1:567487919764:key/dc358503-4ed1-4857-aa04-8462d3208eca"
12     }
13   ]
14 }
```

DevopsDemoSQSLeastPrivilege [Info](#)

Least-privilege access to devops-demo-queue for sending and receiving messages.

Edit

Delete

Policy details

Type	Creation time	Edited time	ARN
Customer managed	September 22, 2026, 16:00 (UTC+05:30)	September 22, 2026, 16:00 (UTC+05:30)	<code>arn:aws:iam::567487919764:policy/DevopsDemoSQL-eastPrivilege</code>

Permissions

Entities attached

Tags

Policy versions (1)

Last Accessed

Permissions defined in this policy [Info](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it.

Copy

Edit

Summary

JSON

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "SQSAccessToDevopsDemoQueue",
6       "Effect": "Allow",
7       "Action": [
8         "sqs:GetQueueUrl",
9         "sqs:GetQueueAttributes",
10        "sqs:SendMessage",
11        "sqs:ReceiveMessage",
12        "sqs:DeleteMessage"
13      ],
14      "Resource": "arn:aws:sqs:us-east-1:567487919764:devops-demo-queue"
15    }
16  ]
17 }
```

```

$ aws sts get-caller-identity
{
  "UserId": "AIDAYIIHK02KKZ0Z4X0BZ",
  "Account": "567487919764",
  "Arn": "arn:aws:iam::567487919764:user/DevopsSQSOperator"
}

$ aws sts assume-role \
  --role-arn arn:aws:iam::567487919764:role/DevopsDemoSQSRole \
  --role-session-name SQS-Security-Lab
{
  "Credentials": {
    "KeyId": "ASIAVYIIHK02KXUV6VSL",
    "SecretAccessKey": "N2l1t7k/vza3Cmu335arVdcyQFdv9is/dZc",
    "SessionToken": "IQo3b3p221ux2VjEzJz////////wEACXVzLWVhc3QTM5JMEYCIQD9n1YUw4XDwBk1b2cBTPK60pdrKkgB1Ysw1J0ee/QiHwMbv8n/tHxkZPn/n3Yi41813//QNrXRbTVFM449ukqYCCkT////////wEQABOMNTY3Nd30TE5NzYw8wYUWk0b31XUMfBA513TCGXzvhmKv6FwPvHAdJgkNo/dzgbc0thnpq87czedn1fJwK62mdgrgsmuabYhr/ABVcZL6g/AR491cPCwEILPUDqy4cLcYmhZ1ZKcL5uVd85setZLElPuhNOA3Jd1ym3j1p0REEGT3XMP09vNw3jEfvuzY7p3BHzqQz-jR8a2MhnydyG0wK74j0pAqAw9cMkUQ97czBqumq4E5Qz180WQD20c13p/dUvKcwcy/1rMuPwK2X0Dxzc07dxcD5FFuH/Ynerbi4d0W+dxc10G0fC3D8rs+e81lWg8pkyCq1E0QZwdaQznob84UfFP0YQZAtgPwHR11Yv",
    "Expiration": "2026-09-22T12:17:42:00:00"
  },
  "AssumedRoleUser": {
    "AssumedRoleId": "AR0AVYIIHK02K0ESCFDDM:SQS-Security-Lab",
    "Arn": "arn:aws:sts::567487919764:assumed-role/DevopsDemoSQSRole/SQS-Security-Lab"
  },
  "PackedPolicySize": 18
}

```

[illegible]

```

$ ./swla Snehad8080E0C107F17AS MINGW64 /
$ export AWS_SESSION_TOKEN=IQoJb3JlbnR1X2VhbnR//////////wEaCXVzLWVhc3QzM5IMEYCIQDkBM1aen9w0bJEDoE/RawbXQjDFngLkCpw17F5GFD+Q1hA76tqXf/FOESYQNILbuSrD9x0579v5fW1G0s17RkYCKK//////////wE
tAoB8R1Vub17550kDAQJfYm9rcCh1pN1n6Z8D0u0W1E21aaEaQj6KvN2L724a477q3R8y0v/g56f0I7Q30xbt1ah5e0wS:Ys13L9D/4CMA/41Qr706+53021L7A4t3C1PERM1D52HtBme1HxAVX3FRK2Guzm0E0+Hm5y
Bum180Yk8Hmny7v0tW5sGFpMpb4r0dG0wBx3jY5dcMAOPd/qDnVbL4w4kmtC/Dxk8R3c9B847GDfUvDhV2dZnAEH6GfY4TbYhXzqj/ZKLLP1u3C5vCRv0tNY1pZf2cc88u5wG6G/xh4YfK5Hw6U3X34rFPD1CvI0Z0E0+Hm5y

```

```

$ aws sts get-caller-identity
{
  "UserId": "AROAIYHKO2K0ESCFDDXM:SQS-Security-Lab",
  "Account": "567487919764",
  "Arn": "arn:aws:sts::567487919764:assumed-role/DevopsDemoSQSRole/SQS-Security-Lab"
}

$ aws sts get-caller-identity
{
  "UserId": "AROAIYHKO2K0ESCFDDXM:SQS-Security-Lab",
  "Account": "567487919764",
  "Arn": "arn:aws:sts::567487919764:assumed-role/DevopsDemoSQSRole/SQS-Security-Lab"
}

```

```
$ samala.sneha@DESKTOP-KOF17A3 MINGW64 /
$ aws sqs send-message \
--queue-url "https://sqs.us-east-1.amazonaws.com/567487919764/devops-demo-queue" \
--message-body '{"orderId":"LAB8-001","customer":"Sunith","amount":1500,"status":"SECURITY-TEST"}' \
--region us-east-1
{
  "MD5OfMessageBody": "8154b1446a8aaf2fab232d054f38f0b1",
  "MessageId": "aefc1e4a-5c95-44d8-9e3e-b72760c94889"
}
```

```
Sama1a Sneha@DESKTOP-K0F17A5 MINGW64 /S
```

```

sneha@SnehaDESKTOP-KGF1765 WING64 /
$ aws sqs receive-message
> --queue-url "https://sqs.us-east-1.amazonaws.com/567487919764/devops-demo-queue" \
> --max-number-of-messages 1 \
> --visibility-timeout 30 \
> --region us-east-1
{
  "Messages": [
    {
      "MessageId": "ae1c1e4a-5c95-44d8-9e3e-b72760c94889",
      "ReceiptHandle": "AQEBH4P6tEccGm/32LmVtv2MvNrOnI2hyac1MrQGxULDoo5GSGQCy0XyuuF1XEQ1eGccv13NwJ77EM8DmW2Uxn1uv2YHGicXj2wE8RZrjKwhP2HwKANIzXxyYJ2NK/1Gczoh/xjIPU4qHdebr3x510B3mn60wR2NyNL/8U7ZF9crlP8l0LQDp6gW3CccZ12Zk1t0vUtlP2tFw/NKplhA1yS2B1tEuSQ1YKtV/c52MU0oqBmGmOmXq2e3t5t1cQaU0bC7eYeaaq113jYvPbKhBRHsVF3BV/cAhgUR0FmPwIb1q7w5H9wC87kdb88F5x5e14uXZhrRA==",
      "Body": "{\"orderId\":\"LAB8-001\",\"customer\":\"Sunith\",\"amount\":1500,\"status\":\"SECURITY-TEST\"}"
    }
  ]
}
sneha@SnehaDESKTOP-KGF1765 WING64 /
$

```

22. Conclusion

In this lab, security was implemented for the Amazon SQS queue devops-demo-queue.

A customer-managed AWS KMS key named devops-sqs-key was created and configured for SSE-KMS encryption. A least-privilege SQS IAM policy and a KMS usage policy were created and attached to the dedicated DevopsDemoSQSRole.

A non-root IAM user, DevopsSQSOperator, was configured to assume the role through AWS STS. The assumed-role identity was successfully verified from Git Bash.

The SendMessage test successfully delivered a message to the encrypted SQS queue, proving that the IAM, SQS, and KMS configuration is functioning.

The remaining ReceiveMessage, DeleteMessage, and intentional AccessDenied tests can be performed as additional validation of the completed security configuration.

This lab demonstrates the practical implementation of:

IAM + Least Privilege + IAM Roles + KMS + SSE-KMS + Amazon SQS Security.

LAB 9 — Amazon MQ for RabbitMQ

1. Objective

To provision and validate an Amazon MQ broker using RabbitMQ, access the RabbitMQ Management UI, create a durable queue, publish a persistent message, retrieve the message, and verify the broker, queue, connection, and channel states.

2. Purpose

The purpose of this lab is to understand the basic operation of Amazon MQ for RabbitMQ and the RabbitMQ messaging workflow.

The practical exercise demonstrates:

- Amazon MQ RabbitMQ broker creation
- RabbitMQ Management UI access
- Queue creation
- Durable queue configuration
- Message publishing
- Persistent message delivery
- Message retrieval
- Queue-state verification
- Connection verification
- Channel verification

3. Prerequisites

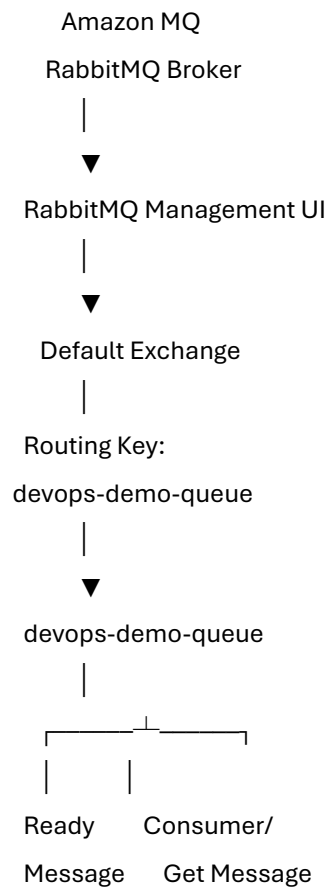
The following were required:

- AWS account
- AWS Management Console access

- Amazon MQ permissions
 - Web browser
 - RabbitMQ Management UI credentials
 - AWS Region: us-east-1
-

4. Architecture / Message Flow

The lab follows this messaging flow:



The test sequence was:

Create Broker

↓

Access Management UI

↓

Create Queue

↓

Publish Message

↓

Verify Message

↓

Retrieve Message

↓

Verify Queue State

↓

Check Connections

↓

Check Channels

5. Broker Configuration

The Amazon MQ RabbitMQ broker used for the lab was:

Configuration	Value
Broker name	devops-rabbitmq-broker
Broker ID	b-49472548-1038-4ba0-a81d-114c7044ae01
Region	us-east-1
Engine	RabbitMQ
RabbitMQ version	4.3.5
Deployment	Single-instance
Instance type	mq.m7g.medium
VPC	vpc-0fff3eb0730a112f0
Subnet	subnet-0e7dc7ba62cb82e14
Accessibility	Publicly accessible
AMQP protocol	AMQPS
AMQP port	5671
Prometheus port	16001

The broker was successfully created and reached the Running state.

6. RabbitMQ Management UI

The RabbitMQ Management UI was accessed after the broker became available.

The management interface provided access to:

- Overview
- Connections
- Channels
- Exchanges
- Queues and Streams
- Administration

The RabbitMQ Management UI displayed:

- RabbitMQ version: 4.3.5
- Erlang version: 27.3.4.2
- Cluster: devops-rabbitmq-broker
- Virtual host: /
- Management user: rabbitadmin

7. Queue Creation

A queue named:

devops-demo-queue

was created.

Queue configuration

Setting	Value
Queue name	devops-demo-queue
Queue type	Classic
Durable	Yes
Auto-delete	Disabled
Arguments	Default / empty

The queue was successfully created and appeared in the RabbitMQ Management UI.

8. Message Publishing

A test message was published to the queue.

Publishing configuration

Setting	Value
Exchange	(AMQP default)
Routing key	devops-demo-queue
Delivery mode	2 — Persistent
Payload encoding	String
Headers	Empty
Properties	Empty

Message payload

```
{  
  "orderId": "RABBIT-001",  
  "customer": "Sunith",  
  "amount": 2500,
```

```
"status": "NEW"
}
```

The message was successfully published.

After publishing, the queue showed one Ready message.

9. Message Verification

The published message was verified in the RabbitMQ Management UI.

The message displayed:

- Exchange: (AMQP default)
- Routing key: devops-demo-queue
- Delivery mode: 2
- Payload size: approximately 95 bytes
- Payload containing the expected JSON order data

This confirmed that the message reached the intended RabbitMQ queue.

10. Message Retrieval

The RabbitMQ Management UI's **Get messages** operation was used to retrieve the test message.

The retrieved message contained:

```
{
  "orderId": "RABBIT-001",
  "customer": "Sunith",
  "amount": 2500,
  "status": "NEW"
}
```

The Management UI displayed a warning that getting messages from a queue is a destructive action.

The message retrieval was therefore treated as a verification operation and was not unnecessarily repeated.

Screenshot:

MQ-RABBIT-08-RabbitMQ-Message-Consume.png

11. Final Queue Verification

The RabbitMQ Overview subsequently showed the following current queue state:

Metric	Value
Ready	1
Unacked	0
Total	1
Consumers	0

Therefore, the final observed state at the time of documentation was:

Ready = 1

Unacked = 0

Total = 1

Consumers = 0

This state should be recorded as the actual observed Management UI state rather than assuming that the queue is empty.

12. Connections Verification

The RabbitMQ Overview reported:

Connections: 0

The Connections section was checked to verify that there were no active AMQP client connections.

No connections were closed or modified during verification.

Screenshot:

MQ-RABBIT-10-Connections.png

13. Channels Verification

The RabbitMQ Overview reported:

Channels: 0

The Channels section was checked to verify that there were no active AMQP channels.

No channels were closed or modified during verification.

Screenshot:

MQ-RABBIT-11-Channels.png

14. RabbitMQ Overview Verification

The final Overview page showed:

Category	Observed value
Ready messages	1
Unacked messages	0
Total messages	1
Consumers	0
Connections	0
Channels	0
Exchanges	8
Queues	1

The broker node displayed:

rabbit@localhost

The Management UI reported RabbitMQ 4.3.5 with Erlang 27.3.4.2.

Screenshot:

MQ-RABBIT-12-RabbitMQ-Overview-Final.png

15. Verification Summary

Test	Result
Broker creation	Passed
Broker reached Running state	Passed
RabbitMQ Management UI access	Passed
Queue creation	Passed
Classic queue configuration	Passed
Durable queue configuration	Passed
Message publishing	Passed
Persistent delivery mode	Verified
Message payload	Verified
Message retrieval	Passed
Queue state verification	Completed
Connections verification	Completed
Channels verification	Completed

16. Screenshots / Evidence

The following screenshots should be retained as lab evidence:

Screenshot	Purpose
MQ-RABBIT-05-RabbitMQ-Management-UI.png	RabbitMQ Management UI
MQ-RABBIT-06-RabbitMQ-Queue.png	Queue configuration
MQ-RABBIT-07-RabbitMQ-Message-Publish.png	Published message
MQ-RABBIT-08-RabbitMQ-Message-Consume.png	Retrieved message
MQ-RABBIT-09-Queue-Final-State.png	Queue state
MQ-RABBIT-10-Connections.png	Connections verification
MQ-RABBIT-11-Channels.png	Channels verification
MQ-RABBIT-12-RabbitMQ-Overview-Final.png	Final broker overview

17. Observations

1. Amazon MQ successfully provisioned a managed RabbitMQ broker.
2. The RabbitMQ Management UI provided administrative visibility into queues, exchanges, connections, and channels.

3. A durable Classic queue was successfully created.
 4. The default AMQP exchange was used with the queue name as the routing key.
 5. Delivery mode 2 was used to publish the test message persistently.
 6. The JSON payload was successfully published and subsequently retrieved.
 7. The Get Messages operation is destructive and therefore should be used carefully.
 8. The final Overview showed one Ready message, zero Unacked messages, and zero Consumers.
 9. No active connections or channels were present during the final verification.
 10. The broker was running as a single-instance mq.m7g.medium deployment.
-

18. Troubleshooting

Broker not available

Verify that:

- The Amazon MQ broker status is Running.
- The correct AWS Region is selected.
- The broker endpoint is correct.
- Network access is available.

RabbitMQ Management UI unavailable

Verify:

- Broker status.
- Management endpoint.
- Network connectivity.
- Correct username and password.

Queue not visible

Verify:

- Correct virtual host /.
- Correct queue name.
- Correct RabbitMQ broker.
- Queue creation completed successfully.

Message not visible

Verify:

- Correct exchange.
- Correct routing key.
- Queue name.
- Message publication result.

- Queue Ready count.

Message retrieval behavior

The RabbitMQ Management UI warns that getting messages from a queue is destructive. Avoid repeatedly clicking **Get messages** when the purpose is only to inspect the queue.

19. Security and Operational Considerations

- RabbitMQ credentials should be kept confidential.
 - Management access should be restricted to authorized users.
 - Public accessibility should be used only when required for the lab.
 - Production RabbitMQ deployments should use appropriate network controls and security groups.
 - Persistent messages should be used when message durability is required.
 - Administrative operations such as message retrieval and queue deletion should be performed carefully.
 - Temporary lab infrastructure should be removed after the exercise to avoid unnecessary AWS charges.
-

20. Conclusion

The Amazon MQ for RabbitMQ lab successfully demonstrated the fundamental RabbitMQ messaging workflow using a managed AWS service.

A single-instance RabbitMQ broker was provisioned, the RabbitMQ Management UI was accessed, and a durable Classic queue named devops-demo-queue was created. A persistent JSON message was successfully published using the default exchange and the queue name as the routing key.

The message was subsequently retrieved and its payload was verified. The RabbitMQ Management UI was also used to inspect the queue, connections, channels, exchanges, and broker status.

LAB 11 — AWS Secrets Manager — Create & Retrieve Secrets

1. Objective

The objective of this lab is to understand the basic functionality of **AWS Secrets Manager** by creating, securely storing, and retrieving a secret containing application credentials.

The lab demonstrates:

- Creating a generic secret
 - Storing key/value pairs securely
 - Using the AWS-managed Secrets Manager KMS key
 - Configuring a secret name and description
 - Understanding secret rotation
 - Retrieving a secret through the AWS Management Console
 - Verifying IAM access behavior through the AWS CLI
-

2. Purpose

AWS Secrets Manager is a managed service used to securely store and manage sensitive information such as:

- Application usernames and passwords
- Database credentials
- API keys
- OAuth tokens
- Other application secrets

Instead of storing sensitive values directly inside application source code or configuration files, applications can retrieve them from Secrets Manager when required.

Secrets Manager also integrates with AWS Identity and Access Management (IAM) and AWS Key Management Service (KMS) to control access and protect stored secret values.

3. Create the Secret

Step 1 — Open AWS Secrets Manager

1. Sign in to the AWS Management Console.
2. Select the Region **US East (N. Virginia)**.
3. Open **AWS Secrets Manager**.
4. Select **Secrets** from the navigation menu.
5. Click **Store a new secret**.

Initially, there were no secrets configured in the account.

Evidence

Screenshot: SEC-11-01-Secrets-Manager-Empty.png

4. Choose Secret Type

On the **Choose secret type** page:

1. Select **Other type of secret**.
2. Configure the secret using key/value pairs.
3. The secret contains two application credential fields:
 - username
 - password

The password value was entered as a lab-only test value and is intentionally not included in this documentation.

Encryption

The default AWS-managed Secrets Manager KMS key was selected:

aws/secretsmanager

A customer-managed KMS key was not created at this stage because IAM, KMS permissions, and least-privilege configuration are covered in the subsequent Secrets Manager security lab.

Evidence

Screenshot: SEC-11-02-Choose-Secret-Type.png

5. Configure Secret

The secret was configured with the following details:

Setting	Configuration
Secret name	devops/lab11/application
Description	AWS Secrets Manager secret for DevOps Lab 11
Tags	None
Resource permissions	None
Replication	Disabled

No cross-account resource policy was configured.

No replica secret was created in another AWS Region.

Screenshot:

Choose secret type

Secret type [Info](#)

☐ Credentials for Amazon RDS database

☐ Credentials for Amazon DocumentDB database

☐ Credentials for Amazon Redshift data warehouse

☐ Credentials for other database

☐ Managed external secret
Secrets vended by your third-party software vendor.

☒ Other type of secret
API key, OAuth token, other.

Key/value pairs [Info](#)

Key/value

Plaintext

username

devopsuser

Remove

password

LabSecret@2026

Remove

+ Add row

Encryption key [Info](#)

You can encrypt using the KMS key that Secrets Manager creates or a customer-managed KMS key that you create.

aws/secretsmanager

X

🔄

[Add new key](#)

Cancel

Next

You have successfully stored the secret devops/lab11/application. To show it in the list, choose Refresh. Use the sample code to update your applications to retrieve this secret.

View details

See sample code

X

devops/lab11/application

Secret details

Encryption key
[aws/secretsmanager](#)

Secret name
[devops/lab11/application](#)

Secret ARN
[arn:aws:secretsmanager:us-east-1:567487919764:secret:devops/lab11/application-3TKmpA](#)

Secret description
[AWS Secrets Manager secret for DevOps Lab 11](#)

Secret type
-

🔄

Actions

Overview

Rotation

Versions

Replication

Tags

Secret value [Info](#)

Retrieve and view the secret value.

Retrieve secret value

Resource permissions - optional [Info](#)

Add or edit a resource policy to access secrets across AWS accounts.

Edit permissions

Sample code

Use these code samples to retrieve the secret in your application.

[AWS Workload Credentials Provider](#)

EKS add-on

Lambda Extension

Caching Clients

SDK Clients

Overview

Rotation

Versions

Replication

Tags

Secret value [Info](#)

Retrieve and view the secret value.

Close

Edit

Key/value

Plaintext

Secret key

Secret value

username

[devopsuser](#)

password

[LabSecret@2026](#)

6. Configure Rotation

Automatic rotation was intentionally left **disabled**.

The following settings were not configured:

- Rotation schedule
- Rotation window
- Immediate rotation
- Lambda rotation function

Secret rotation will be addressed separately in the dedicated Secrets Manager rotation lab.

7. View Secret Details

The secret devops/lab11/application was opened.

The Secret details page confirmed:

Parameter	Value
Secret name	devops/lab11/application
Encryption key	aws/secretsmanager
Description	AWS Secrets Manager secret for DevOps Lab 11
Region	us-east-1
Secret replication	Disabled
Resource permissions	None

AWS also displayed the secret's ARN.

The actual secret value was not included in this documentation to avoid exposing sensitive information.

8. Key Observations

Observation 1 — Secrets Manager stores sensitive application data

Secrets Manager can be used to centrally store sensitive values rather than embedding them directly in application source code or configuration files.

Observation 2 — Encryption is enabled

The secret was encrypted using the AWS-managed Secrets Manager KMS key:

aws/secretsmanager

Observation 3 — Secret rotation is separate from secret creation

Automatic rotation was disabled during this lab.

Rotation requires additional configuration, including an appropriate rotation mechanism such as a Lambda rotation function depending on the secret type and rotation design.

Observation 4 — IAM controls secret access

Although the secret could be retrieved from the Secrets Manager console, the CLI identity used in the lab was denied access because it lacked:

secretsmanager:GetSecretValue

This demonstrates the importance of IAM authorization for programmatic secret access.

Observation 5 — Least privilege prevents unnecessary access

The existing SQS-specific IAM identity was not granted Secrets Manager permissions simply to make the CLI test succeed.

Instead, the permission boundary was preserved for the dedicated Secrets Manager IAM lab.

9. Security Considerations

The following security practices were followed:

1. Sensitive values were not included in application source code.
2. The secret was stored in AWS Secrets Manager.
3. Encryption was enabled using AWS KMS.
4. Automatic rotation was not enabled until the dedicated rotation lab.
5. No unnecessary cross-account resource policy was created.
6. No cross-Region replica was created.
7. Secrets Manager permissions were not unnecessarily added to the existing SQS IAM identity.
8. The actual password value is excluded from this documentation.

In a production environment, access to secrets should be granted only to the IAM roles, users, or workloads that require them.

Problem: Secret value should not be exposed

Secret values should not be copied into documentation, source code, screenshots, terminal history, or public repositories.

Only record the secret name, metadata, and non-sensitive configuration required for lab evidence.

Conclusion

This lab demonstrated the basic lifecycle of an AWS Secrets Manager secret: creation, secure storage, encryption, console retrieval, and IAM-controlled programmatic access.

The exercise also demonstrated why application workloads should use dedicated IAM permissions rather than broad access to secrets.

The secret will be retained for the upcoming Secrets Manager security and rotation exercises.